EasySoC-Z7010 FPGA/SoC 설계 플랫폼

4. ARM을 이용한 LED, 7-Segment 제어 실습

개요

- ‘FPGA를 이용한 LED, 7-Segment 제어 실습’에서 생성한 7-Segment IP를 Processing System(PS)영역에서 제어하기 위해 AXI4 Peripheral을 생성하고 PS 영역에 연결하여 제어한다.
  - Zynq Z7010의 내부 구성 이해.
  - AMBA 구조와 신호의 동작 이해.
  - AXI4 Peripheral 생성 및 사용법 이해.

HUINS

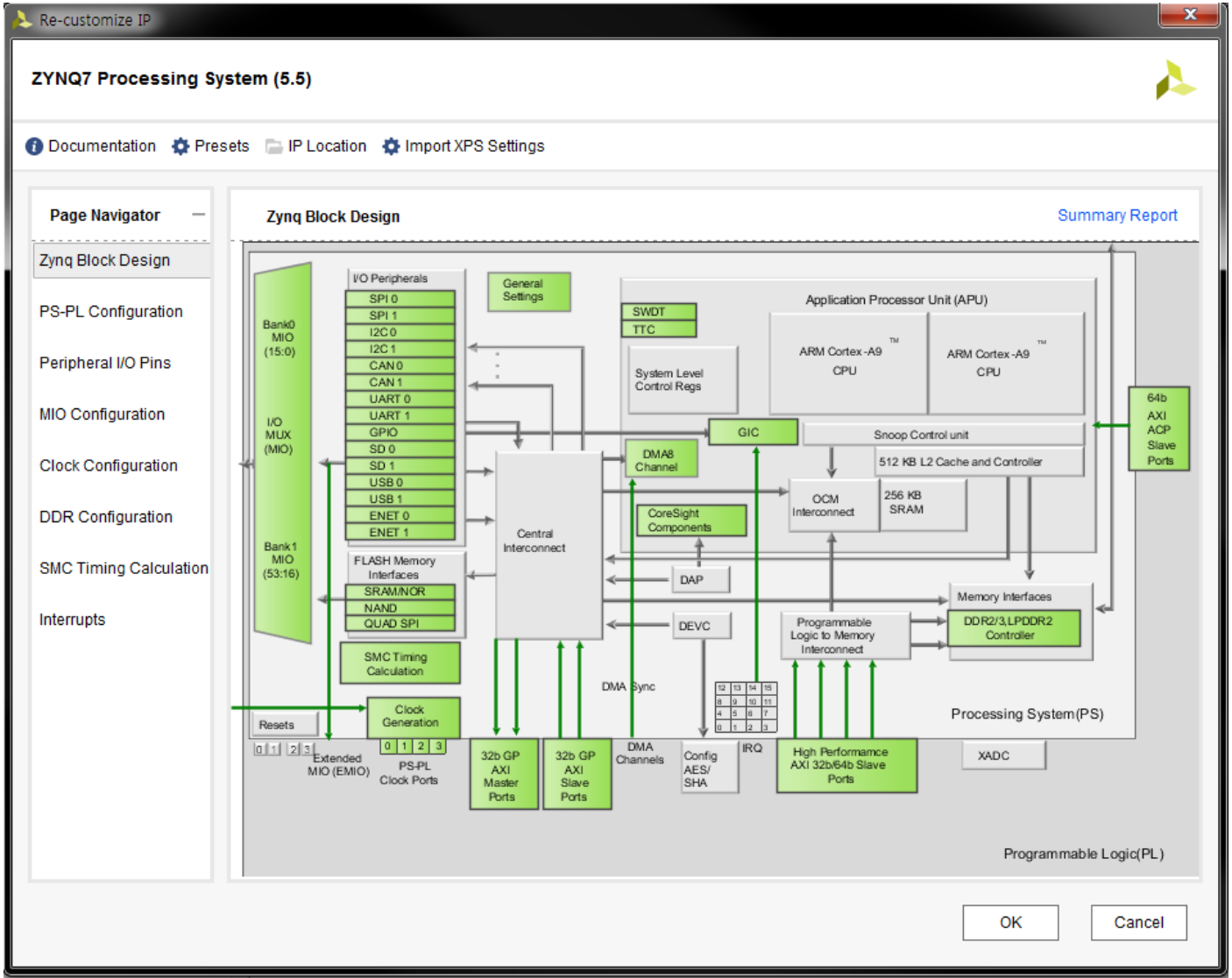
LED와 7-Segment Control IP를 이용한 Zynq 7010 시스템 구조 이해

- 설계 한 PL의 IP를 PS와 연결하기 위해서는 AMBA 프로토콜 규격에 맞는 인터페이스를 설계해야 한다.
- Tool에 의해 생성되는 인터페이스를 이용하여 연결하면서 동작과 방법을 이해 할 수 있다.
- Xilinx Vivado의 경우 임베디드 시스템 설계 환경을 제공한다.
  - GUI(Graphical User Interface) 지원.
  - 시스템을 간단하게 구성이 가능.

HUINS

LED와 7-Segment Control IP를 이용한 Zynq 7010 시스템 구조 이해

- Zynq 7010 시스템 구성



HUINS

LED와 7-Segment Control IP를 이용한 Zynq 7010 시스템 구조 이해

- Zynq 7010 시스템 구성
  - Cortex-A9 Dual Core : Memory Interface, I/O Peripherals, GP AXI Master Interface와 연결
  - Memory Interface : EasySoC-Z7010 모듈은 512MB DDR3 메모리 탑재.
    클릭 시 Memory Controller의 timing 설정이 가능함.
  - I/O Peripherals : GPIO, SPI, I2C, CAN, UART, SD, USB, Ethernet Hard Macro IP.
    클릭 시 Zynq PS에 포함된 IP 사용 유무 체크가 가능함.
  - 추가된 LED/7-Segment IP : 앞서 설계한 IP를 수정하여 AXI-Lite Interface 연결.
  - 다음 그림은 EasySoC-Z7010 Peripheral Memory Map을 나타낸다.

Diagram | Address Editor | top.v

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
processing_system7_0					
Data (32 address bits : 0x40000000 [1G])					
sevenseg_0	S00_AXI	S00_AXI_reg	0x43C0_0000	64K	0x43C0_FFFF

HUINS

LED와 7-Segment Control IP를 이용한 Zynq 7010 시스템 구조 이해

- IP 구성 내용
  - 7-Segment IP 구성 내용
    - ✓ 7-Segment Control Register에 숫자를 입력하여 7-Segment display 실행
    - ✓ 7-Segment Control Register Address : 0x43C0_0000
    - ✓ 0x43C0_0000[15:12] : 4번째 7-Segment의 FND_A~G, FND_DP input 설정
    - ✓ 0x43C0_0000[11: 8] : 3번째 7-Segment의 FND_A~G, FND_DP input 설정
    - ✓ 0x43C0_0000[7: 4] : 2번째 7-Segment의 FND_A~G, FND_DP input 설정
    - ✓ 0x43C0_0000[3: 0] : 1번째 7-Segment의 FND_A~G, FND_DP input 설정

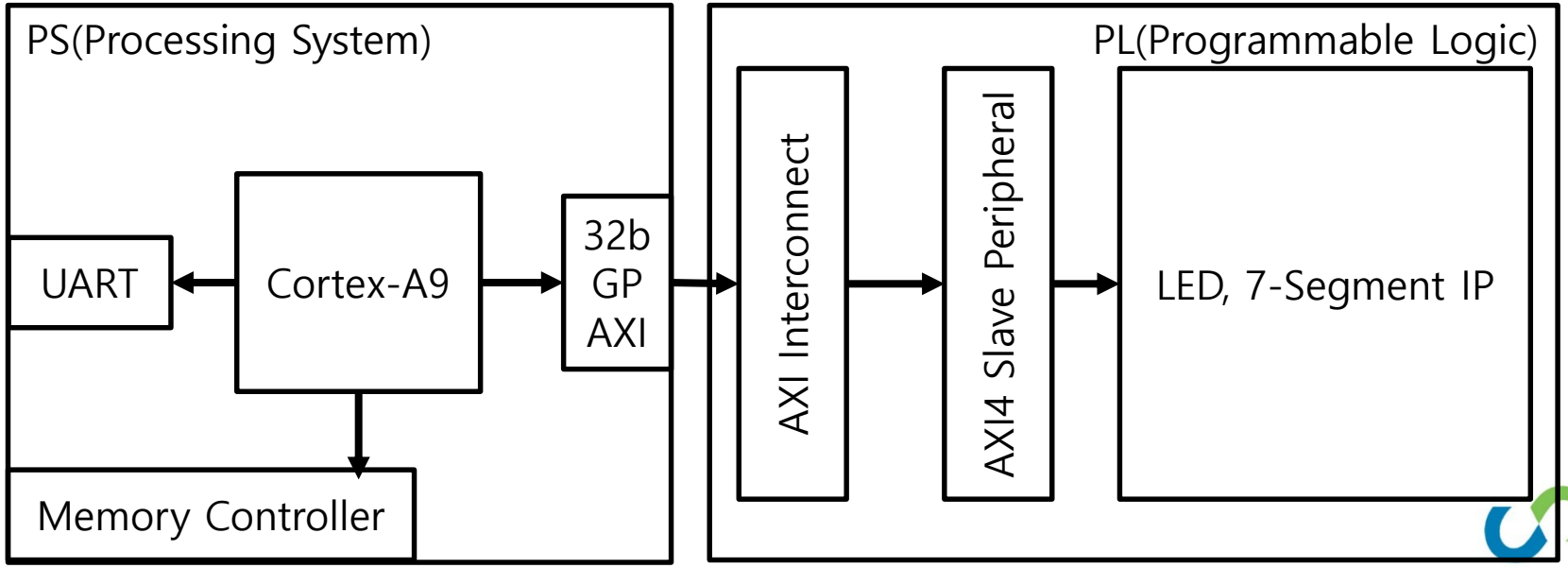
Diagram | Address Editor | top.v

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
processing_system7_0					
Data (32 address bits : 0x40000000 [1G])					
sevenseg_0	S00_AXI	S00_AXI_reg	0x43C0_0000	64K	0x43C0_FFFF

HUINS

LED와 7-Segment Control IP를 이용한 Zynq 7010 시스템 구조 이해

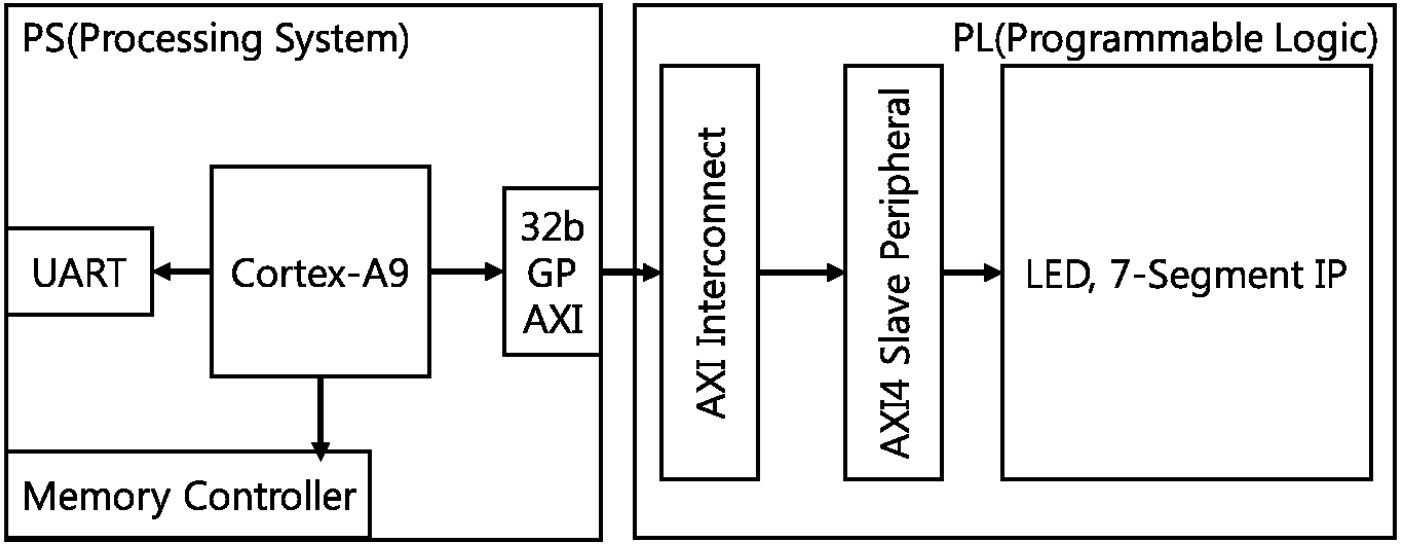
- 디자인 블록 다이어그램
  - Cortex-A9을 이용하여 7-Segment, LED를 제어하기 위한 AXI4-Lite Interface를 생성한다. 이후 Cortex-A9 core로 시스템을 설계한다.
  - 이번 장은 PS(Processor System) 안에 포함되어 있는 Cortex-A9, 32bit GP AXI Master, UART, Memory controller를 사용한다.
    - ✓ 32bit GP AXI Master는 Cortex-A9이 PL에 접근할 수 있도록 함.



HUINS

LED와 7-Segment Control IP를 이용한 Zynq 7010 시스템 구조 이해

- 디자인 블록 다이어그램
  - Top RTL 파일 제작 과정
    - ✓ Vivado에서 그림과 같이 Cortext-A9, UART, 32b GP AXI Master, Memory Controller를 포함한 Processing System을 구성한다.
    - ✓ LED, 7-Segment IP를 제어하기 위한 AXI4 Slave Peripheral을 생성한다.
    - ✓ Processing System과 AXI4 Slave Peripheral IP를 연결해 시스템을 구성하고, 시스템 wrapper HDL 파일을 생성한다.
    - ✓ Top 모듈에서 시스템 Wrapper와 LED, 7-Segment IP를 연결하고 입출력 포트를 선언한다.



HUINS

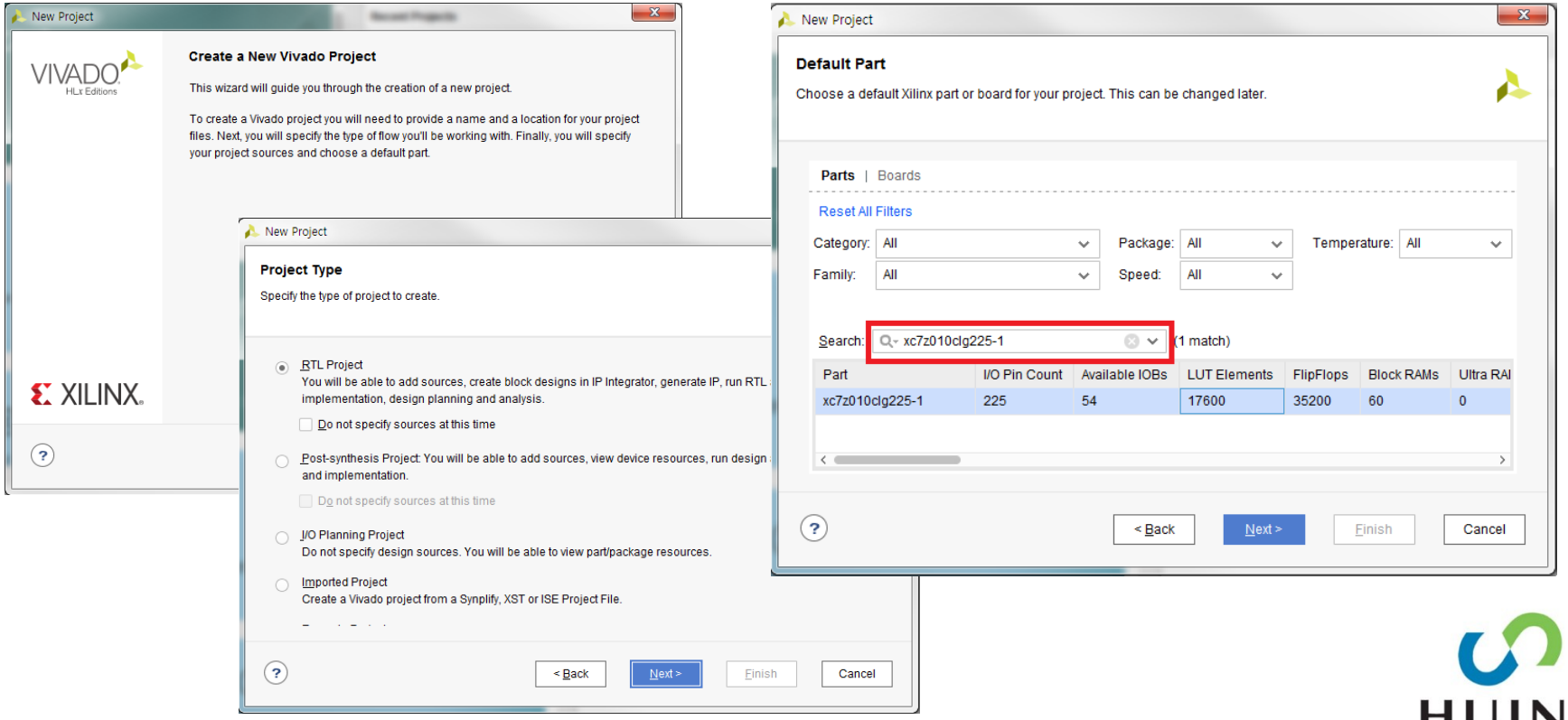
ARM을 이용한 LED, 7-Segment 제어

- 설계 환경
  - 하드웨어 설계 환경
    - ✓ EasySoC-Z7010
    - ✓ Xilinx USB Platform : FPGA download와 Cortex-A9 디버깅
  - Design Language – Verilog HDL
  - 소프트웨어 개발 환경 : Xilinx Design Tools
    - ✓ Vivado
    - ✓ Xilinx Software Development Kit

HUINS

ARM을 이용한 LED, 7-Segment 제어

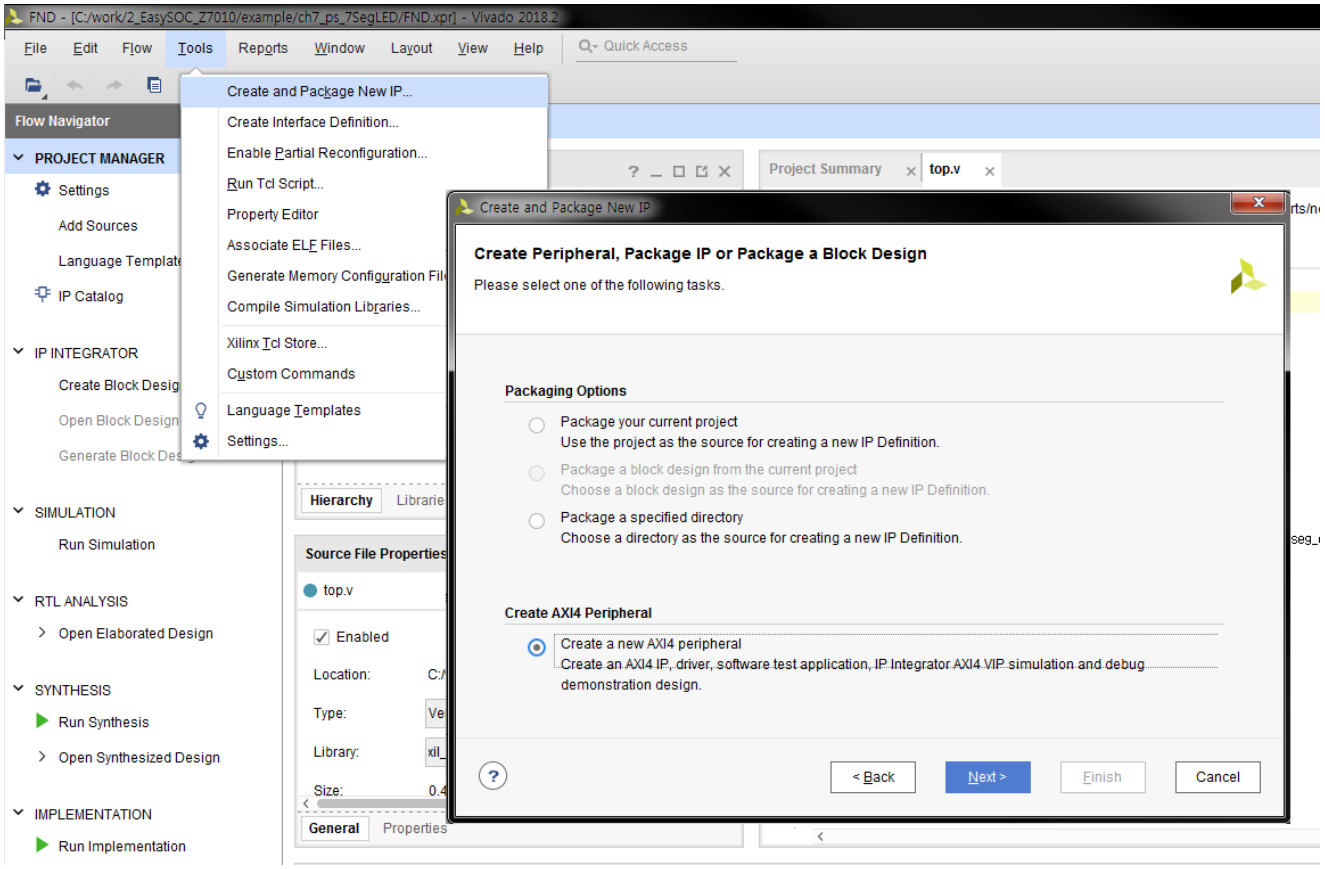
- 실습 진행 과정
  - 프로젝트 생성
    - ✓ 새 프로젝트를 생성한다.
    - ✓ Project type – RTL, Xilinx part – xc7z010clg225-1



HUINS

ARM을 이용한 LED, 7-Segment 제어

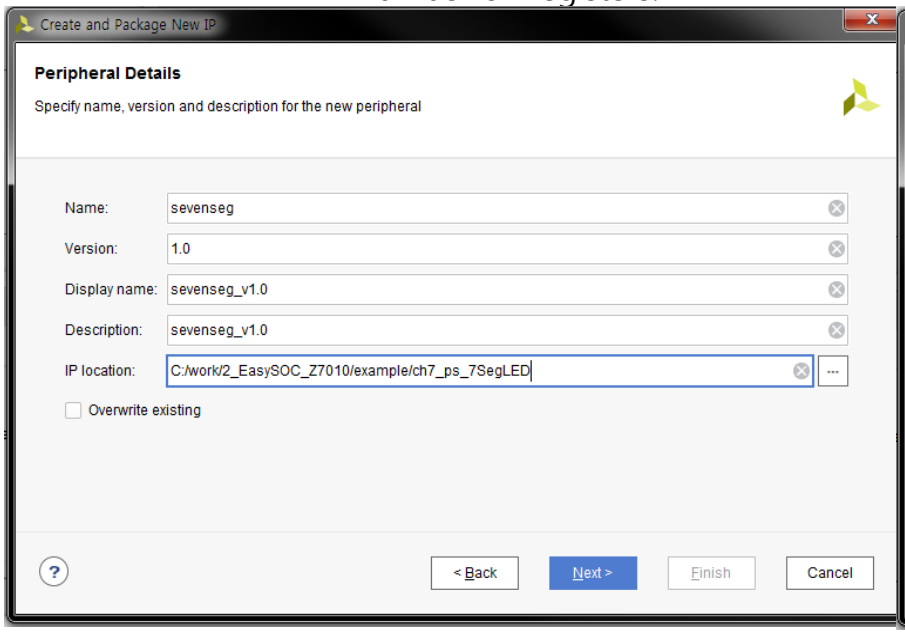
- AXI4 Peripheral 생성
  - ✓ Vivado tool에서 Tools → Create and Package IP 메뉴를 선택한다.
  - ✓ 대화상자에서 Create AXI4 Peripheral을 선택한다.

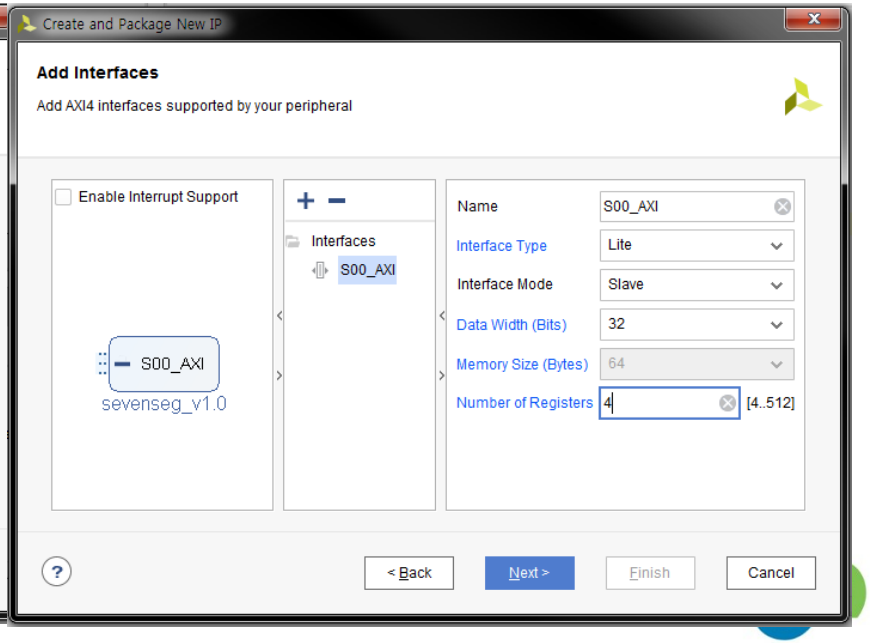


HUINS

ARM을 이용한 LED, 7-Segment 제어

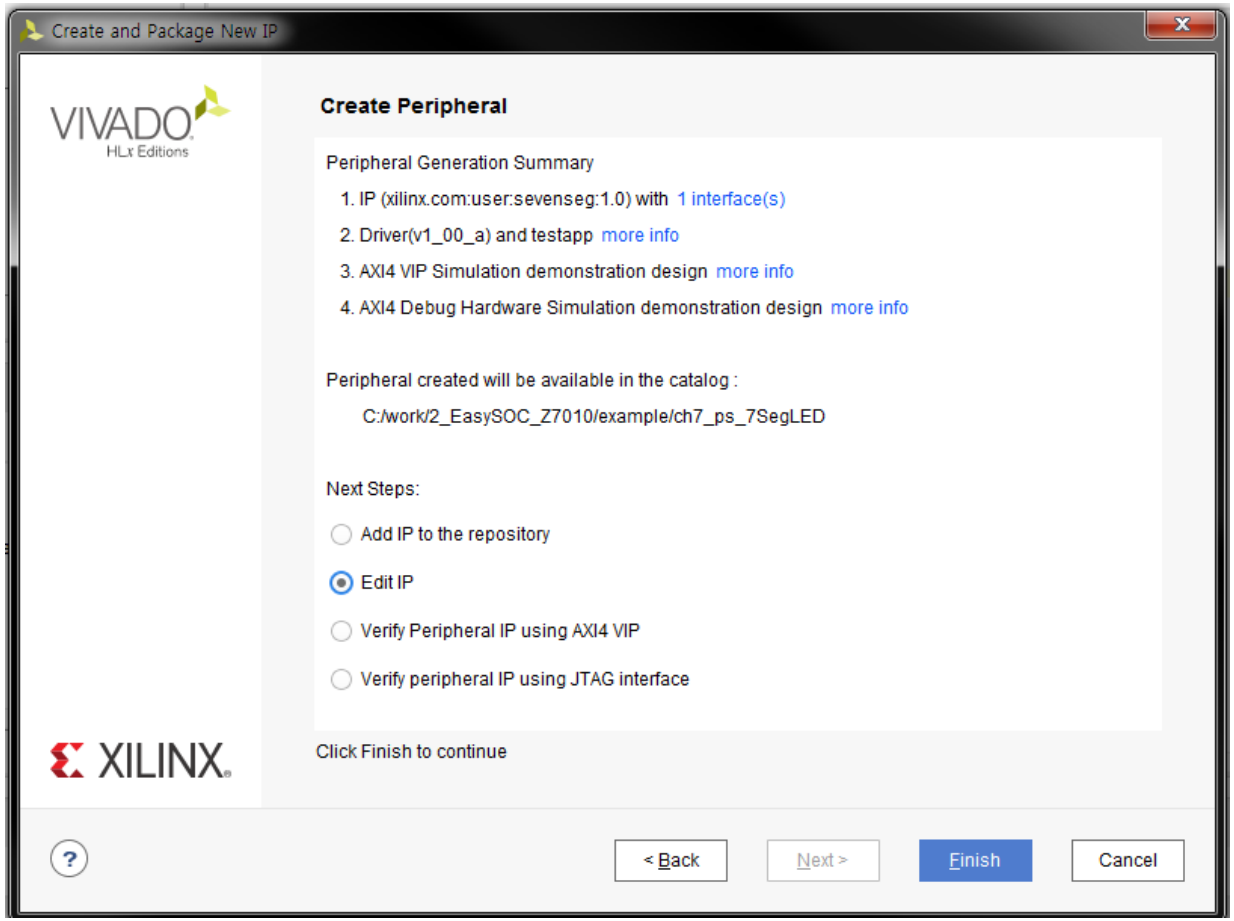
- AXI4 Peripheral 생성
  - ✓ 생성하고자 하는 IP의 이름과 경로를 입력한다.
  - ✓ 인터페이스를 다음과 같이 설정한다.
    - Type: Lite
    - Mode: Slave
    - Data Width: 32
    - Number of Registers: 4





ARM을 이용한 LED, 7-Segment 제어

- AXI4 Peripheral 생성
  - ✓ 생성될 AXI4 Peripheral의 내용을 확인하고, Edit IP를 선택한다.



ARM을 이용한 LED, 7-Segment 제어

- AXI4 Peripheral 생성
  - ✓ edit_sevenseg_v1_0 이라는 이름의 프로젝트가 자동으로 실행된다.
  - ✓ AXI interface instance인 sevenseg_v1_0_S00_AXI.v 파일에서 앞서 설정했던 대로 구성되었음을 확인할 수 있다.
  - ✓ Data Width:32

```
// Width of S_AXI data bus
parameter integer C_S_AXI_DATA_WIDTH = 32,
```

  - ✓ Number of Registers: 4

```
//-- Number of Slave Registers 4
reg [C_S_AXI_DATA_WIDTH-1:0]    slv_reg0;
reg [C_S_AXI_DATA_WIDTH-1:0]    slv_reg1;
reg [C_S_AXI_DATA_WIDTH-1:0]    slv_reg2;
reg [C_S_AXI_DATA_WIDTH-1:0]    slv_reg3;
wire    slv_reg_rden;
wire    slv_reg_wren;
reg [C_S_AXI_DATA_WIDTH-1:0]    reg_data_out;
```

HUINS

ARM을 이용한 LED, 7-Segment 제어

- Processing System에서의 Write명령어 작동.
  - Processing System에서 쓰고자 하는 주소와 데이터를 32b GP AXI Master에 전송한다.
  - 32b GP AXI Master에서 Slave인 AXI Interconnect에 Write 하고자 하는 address(axi_awaddr) 값과, 값이 유효함을 나타내는 address valid(S_AXI_AWVALID)를 인가한다.
  - 32b GP AXI Master에서 Slave인 AXI Interconnect에 Write 하고자 하는 data(S_AXI_WDATA) 값과, 값이 유효함을 나타내는 data valid(S_AXI_WVALID)를 인가한다.
  - Slave인 AXI Interconnect에서 data valid(S_AXI_WVALID)와 address valid(S_AXI_AWVALID) 값을 기반으로 address write ready(axi_awready) 신호를 출력한다.
  - Slave인 AXI Interconnect에서 data valid(S_AXI_WVALID)와 address valid(S_AXI_AWVALID) 값을 기반으로 write ready(axi_wready) 신호를 출력한다.
  - Slave는 address(**axi_awaddr**)에 따라, 해당하는 slave register에 data(S_AXI_WDATA)를 쓴다.

HUINS

ARM을 이용한 LED, 7-Segment 제어

- AXI4 Peripheral 생성
  - ✓ axi_awready, S_AXI_WVALID, axi_wready and S_AXI_WVALID가 모두 입력되었을 경우 slave register write enable 상태가 된다.
  - ✓ Write enable일 경우 write address에 해당하는 slave register에 write data가 쓰여진다.

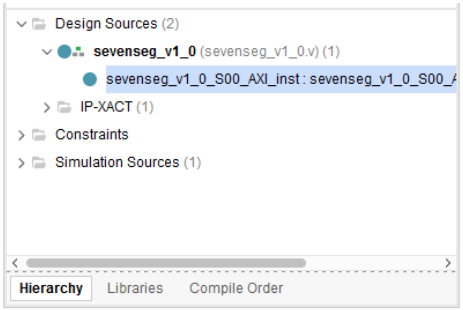
```
// select byte enables of slave registers while writing.
// These registers are cleared when reset (active low) is applied.
// Slave register write enable is asserted when valid address and data are available
// and the slave is ready to accept the write address and write data.
assign slv_reg_wren = axi_wready && S_AXI_WVALID && axi_awready && S_AXI_AWVALID;
```

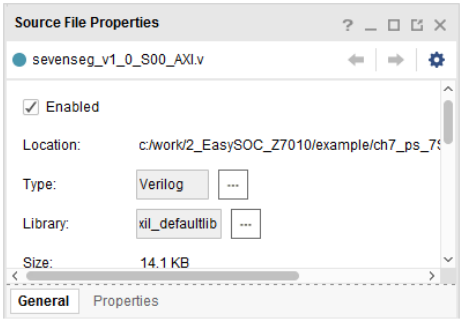
```
    if (slv_reg_wren)
      begin
        case ( axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] )
          2'h0:
            for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
              if ( S_AXI_WSTRB[byte_index] == 1 ) begin
                // Respective byte enables are asserted as per write strobes
                // Slave register 0
                slv_reg0[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
              end
          2'h1:
            for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index = byte_index+1 )
              if ( S_AXI_WSTRB[byte_index] == 1 ) begin
                // Respective byte enables are asserted as per write strobes
                // Slave register 1
```

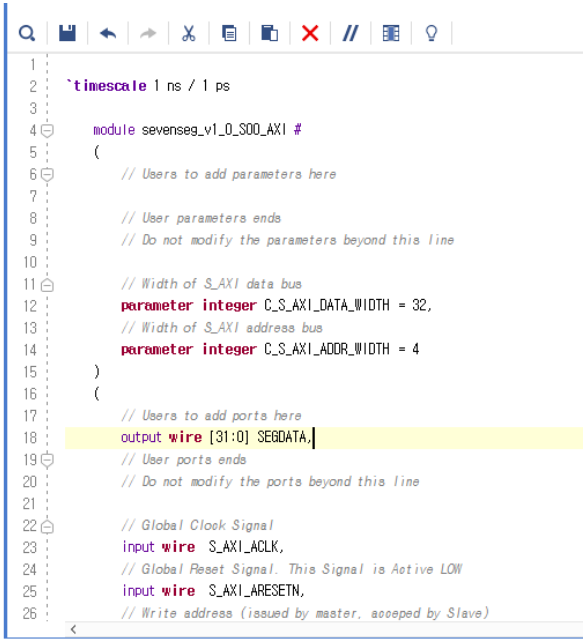
HUINS

ARM을 이용한 LED, 7-Segment 제어

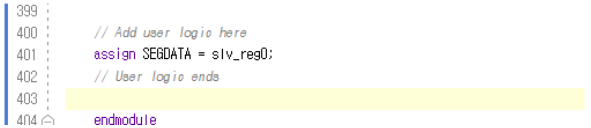
- AXI4 Peripheral 생성
  - ✓ sevenseg_v1_0_S00_AXI.v 파일의 18번 줄에 출력포트 SEGDATA를 선언한다.
  - ✓ PS에서 입력은 slv_reg0에 write되므로(0x43C0_0000) SEGDATA에 slv_reg0를 연결한다.







```
1
2  `timescale 1 ns / 1 ps
3
4      module sevenseg_v1_0_S00_AXI #
5      (
6          // Users to add parameters here
7
8          // User parameters ends
9          // Do not modify the parameters beyond this line
10
11         // Width of S_AXI data bus
12         parameter integer C_S_AXI_DATA_WIDTH = 32,
13         // Width of S_AXI address bus
14         parameter integer C_S_AXI_ADDR_WIDTH = 4
15     )
16     (
17         // Users to add ports here
18         output wire [31:0] SEGDATA,
19         // User ports ends
20         // Do not modify the ports beyond this line
21
22         // Global Clock Signal
23         input wire  S_AXI_ACLK,
24         // Global Reset Signal. This Signal is Active LOW
25         input wire  S_AXI_ARESETN,
26         // Write address (issued by master, acceped by Slave)
```

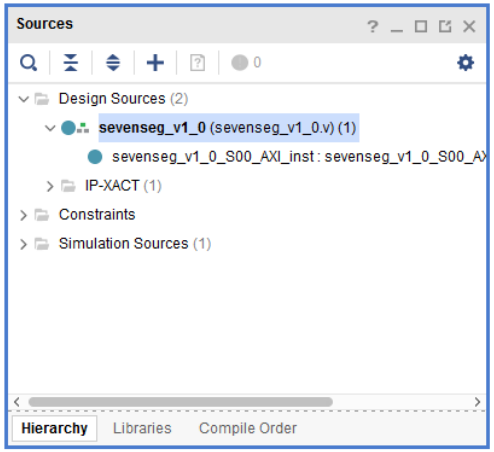


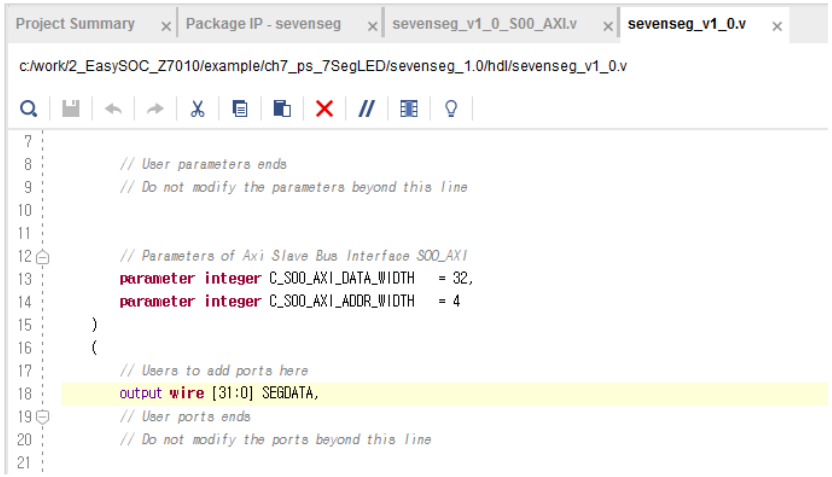
```
399
400     // Add user logic here
401     assign SEGDATA = slv_reg0;
402     // User logic ends
403
404     endmodule
```

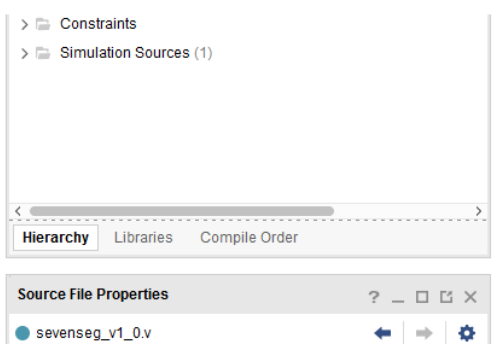
HUINS

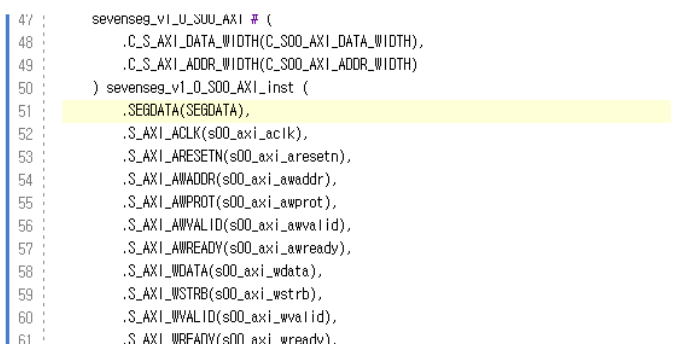
ARM을 이용한 LED, 7-Segment 제어

- AXI4 Peripheral 생성
  - ✓ sevenseg_v1_0.v 파일의 18번 줄에 출력포트 SEGDATA를 선언한다.
  - ✓ AXI instance에서 선언한 출력 SEGDATA와 연결한다.





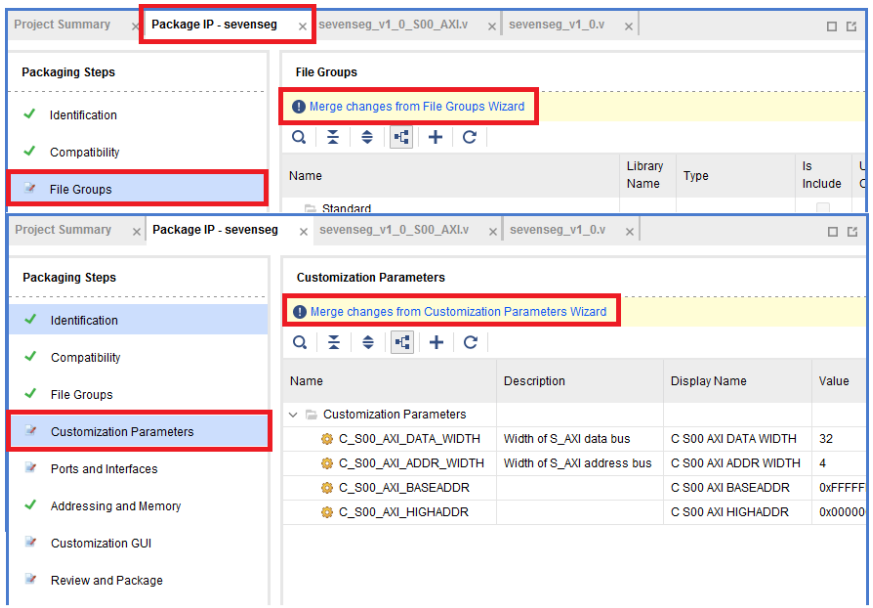


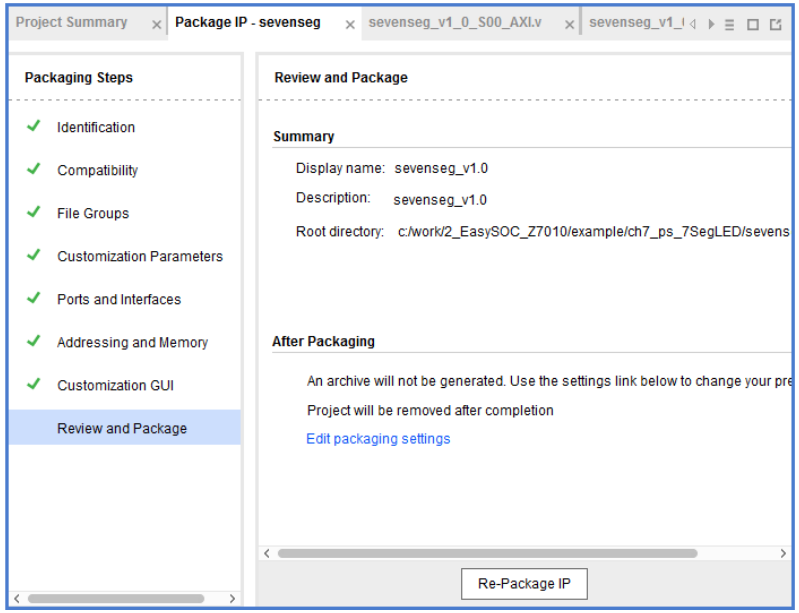


HUINS

ARM을 이용한 LED, 7-Segment 제어

- AXI4 Peripheral 생성
  - ✓ Package IP에서 File Group 탭과 Customization Parameters 탭에서 변경사항을 merge한다.
  - ✓ 지금까지 수정된 IP 사항을 반영하기 위해 Review and Package 탭에서 Re-Package IP 버튼을 클릭한다.

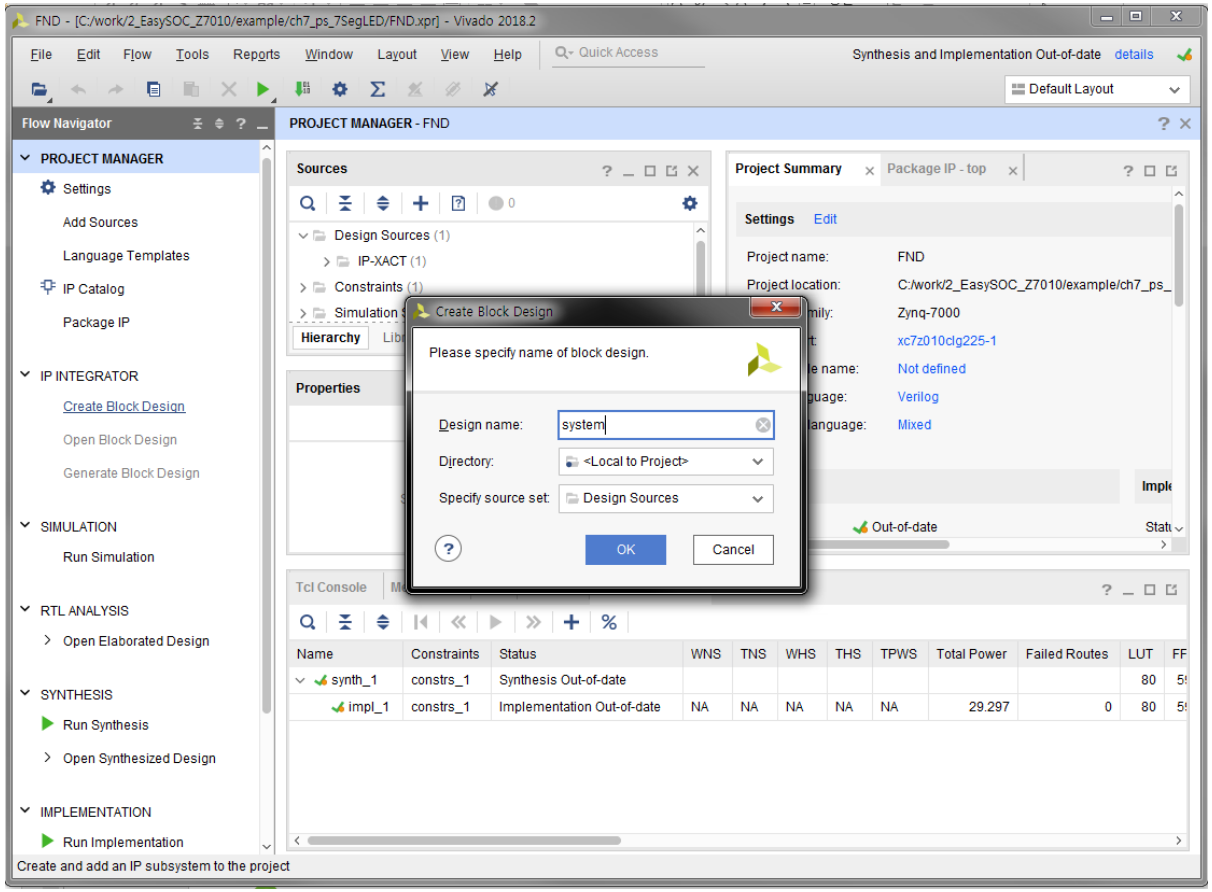




HUINS

ARM을 이용한 LED, 7-Segment 제어

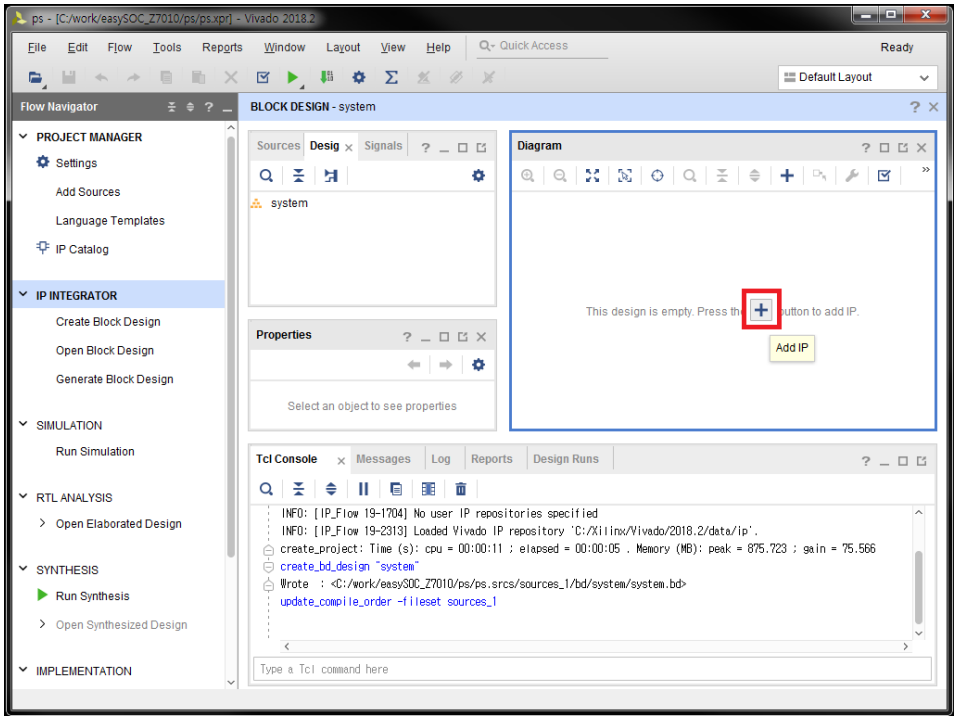
- Processing System 설정
  - ✓ Flow Navigator 창에서 Create Block Design을 클릭해 블록 디자인을 생성한다.

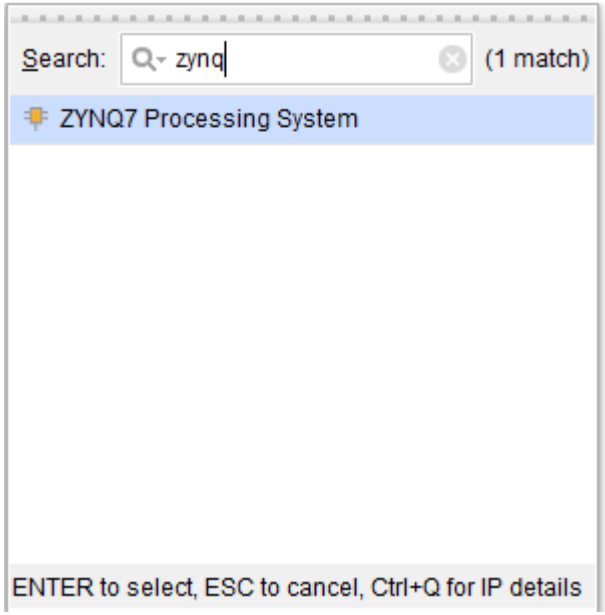


HUINS

ARM을 이용한 LED, 7-Segment 제어

- Processing System 설정
  - ✓ Diagram 서브 창에서 “Add IP”아이콘을 클릭한다.
  - ✓ Xilinx IP를 추가하는 대화상자의 Search 입력란에 “zynq”를 입력하여 ZYNQ7 Processing System 항목을 선택하여 추가한다.

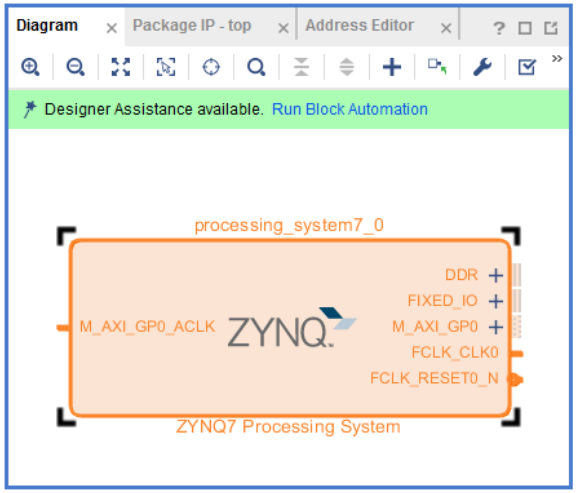


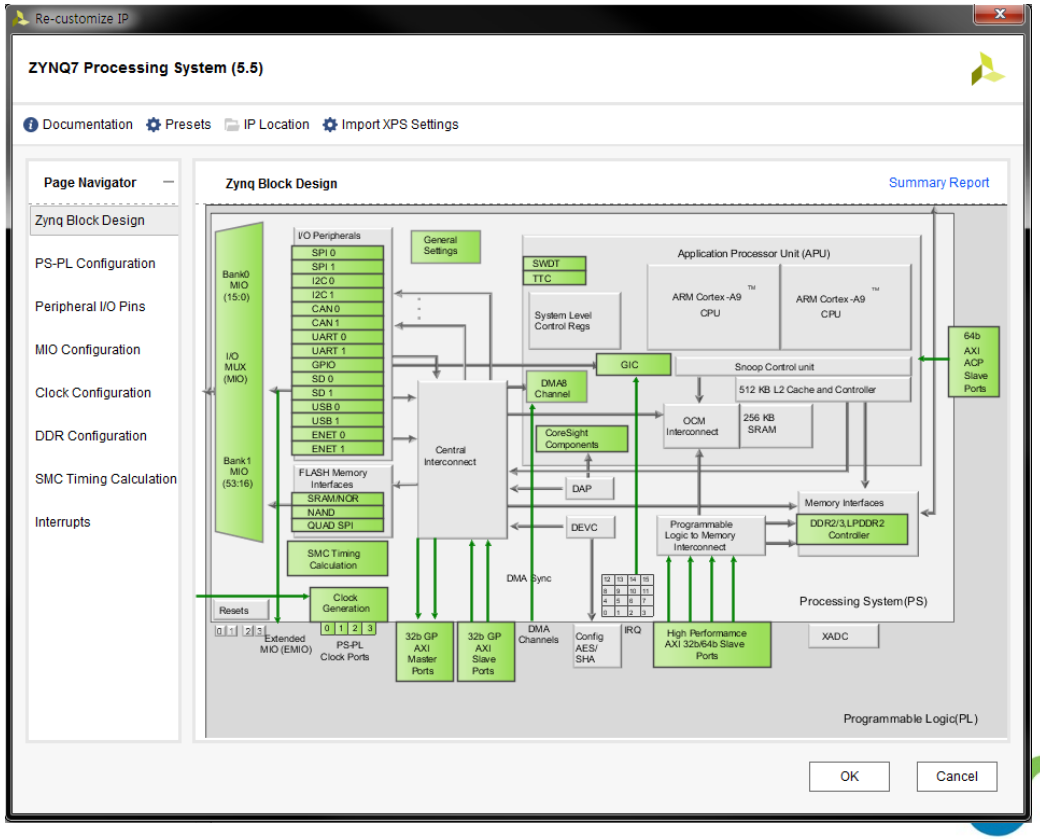


HUINS

ARM을 이용한 LED, 7-Segment 제어

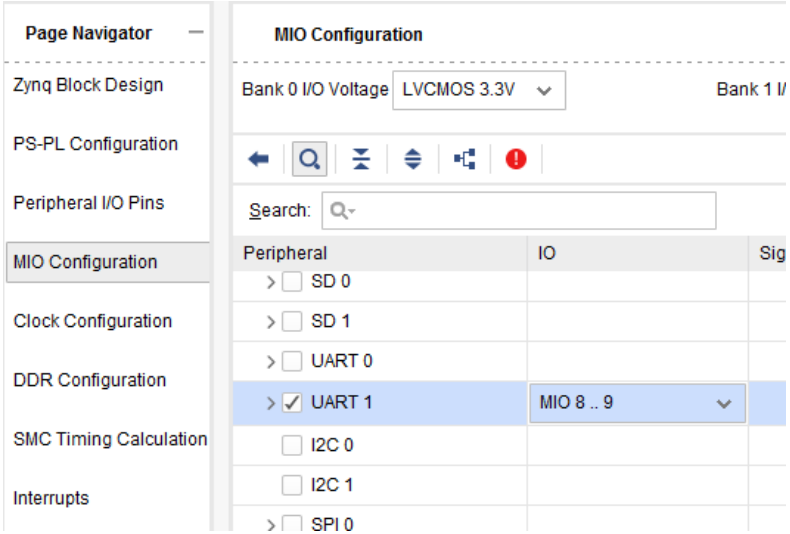
- Processing System 설정
  - ✓ Diagram의 ZYNQ7 Processing System 블록을 더블 클릭해 IP를 수정한다.

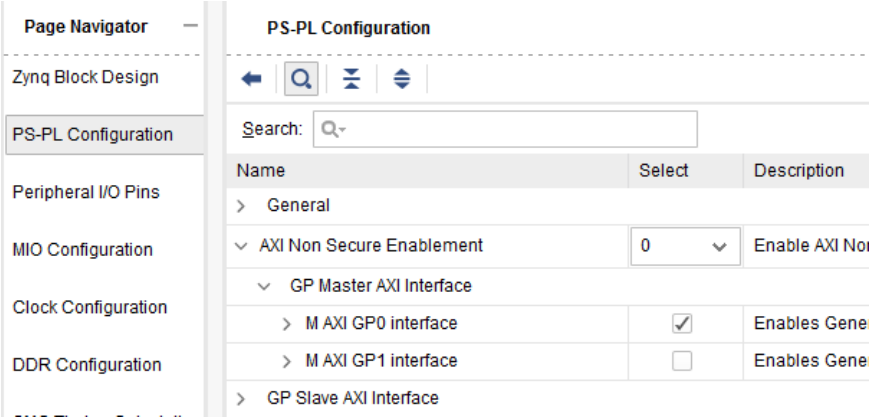




ARM을 이용한 LED, 7-Segment 제어

- Processing System 설정
  - ✓ MIO Configuration 탭에서 UART 1를 Enable하며 할당 핀은 MIO 8, 9로 선택한다.
  - ✓ PS-PL Configuration 탭에서 GP Master AXI Interface – M AXI GP0 interface가 체크되어 있음을 확인한다.

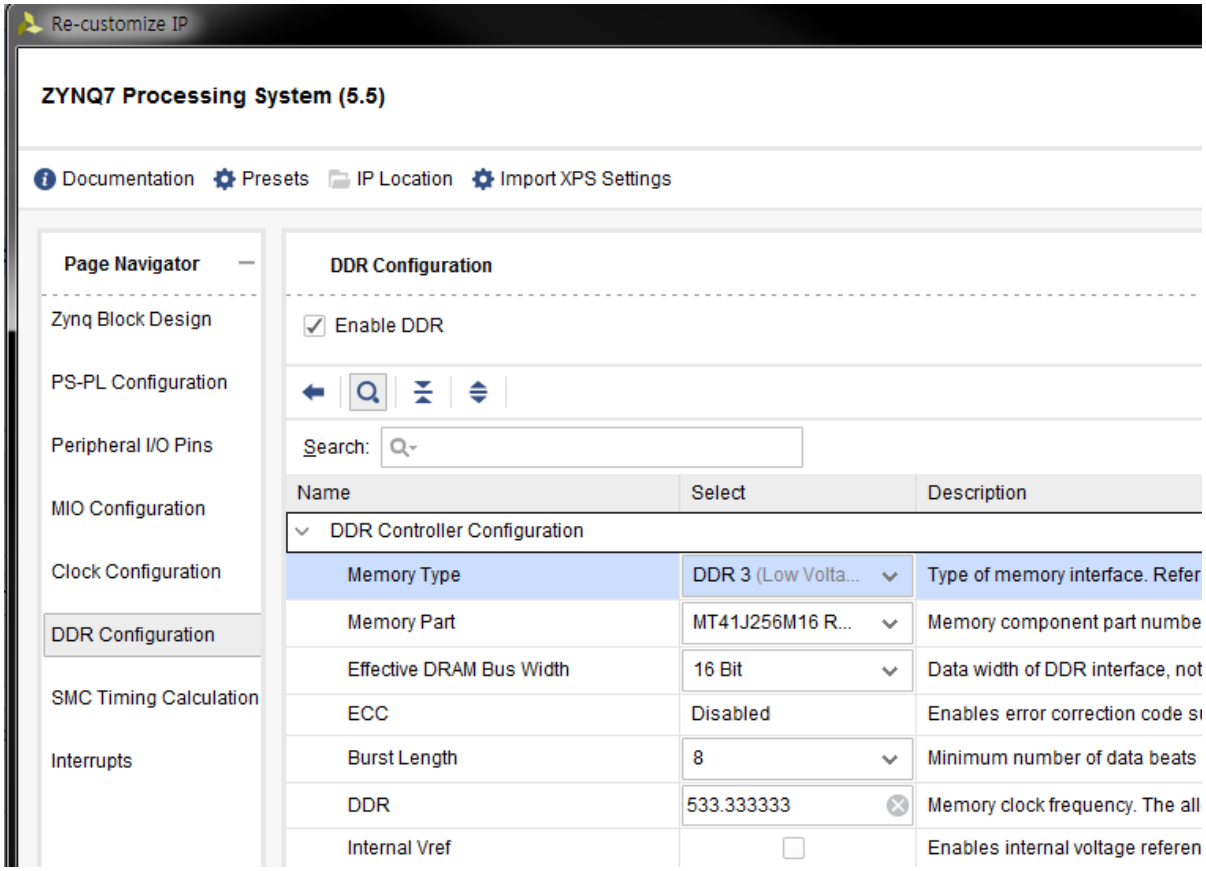




HUINS

ARM을 이용한 LED, 7-Segment 제어

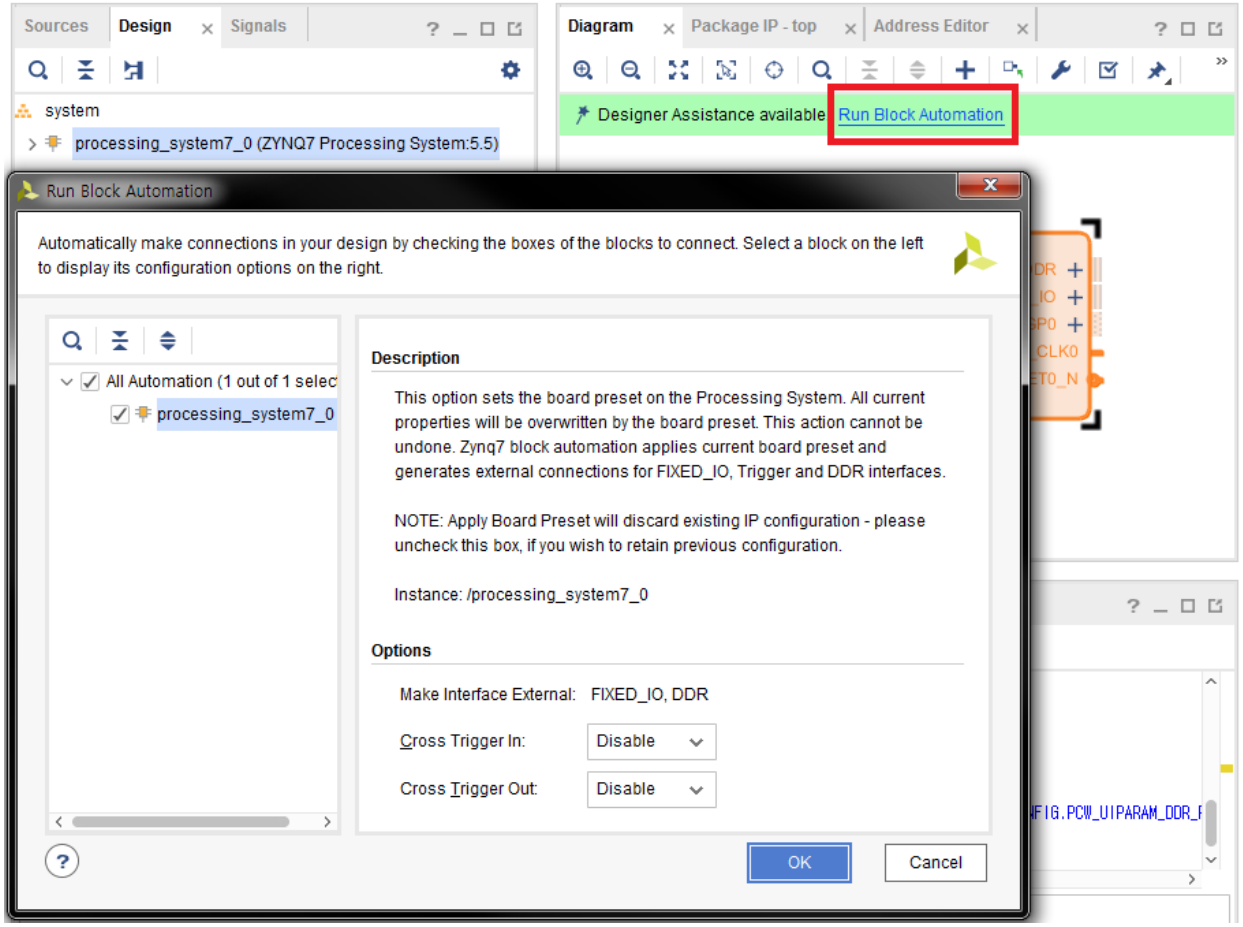
- Processing System 설정
  - ✓ DDR Configuration 탭에서 Memory Type을 DDR3(Low Voltage), Memory Part를 MT41J25616 RE-125를 선택한 뒤 설정을 완료한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

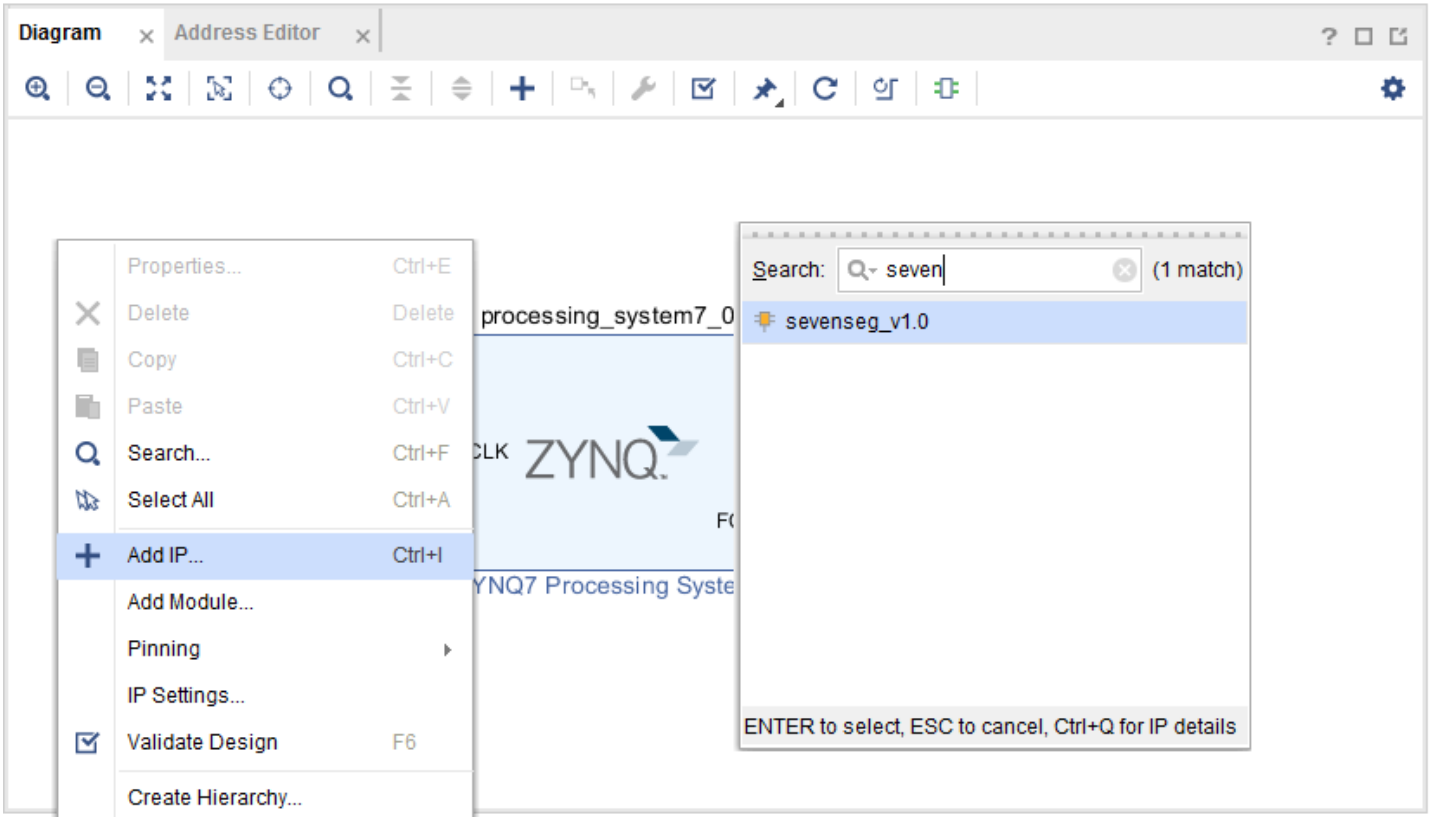
- Processing System과 AXI4 Peripheral 연결
  - ✓ Diagram 창에서 Run Block Automation을 클릭해 DDR, FIXED_IO를 연결한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

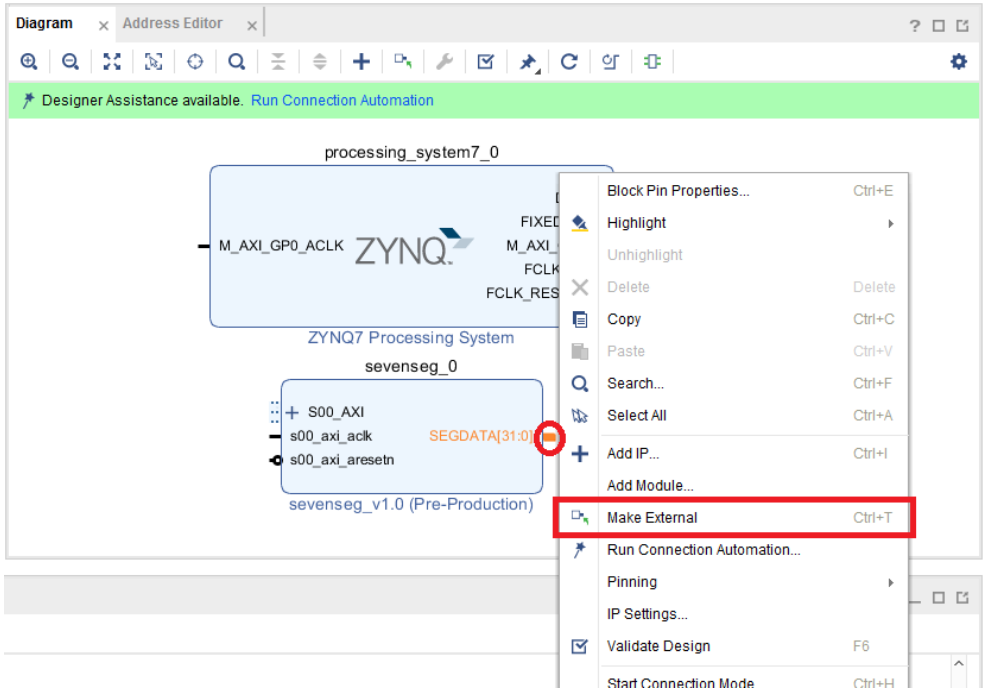
- Processing System과 AXI4 Peripheral 연결
  - ✓ Diagram 창에서 Ctrl + I 를 입력하거나 우클릭하여 Add IP를 클릭한다.
  - ✓ 앞서 생성했던 sevenseg_v1.0 IP를 검색하여 추가한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

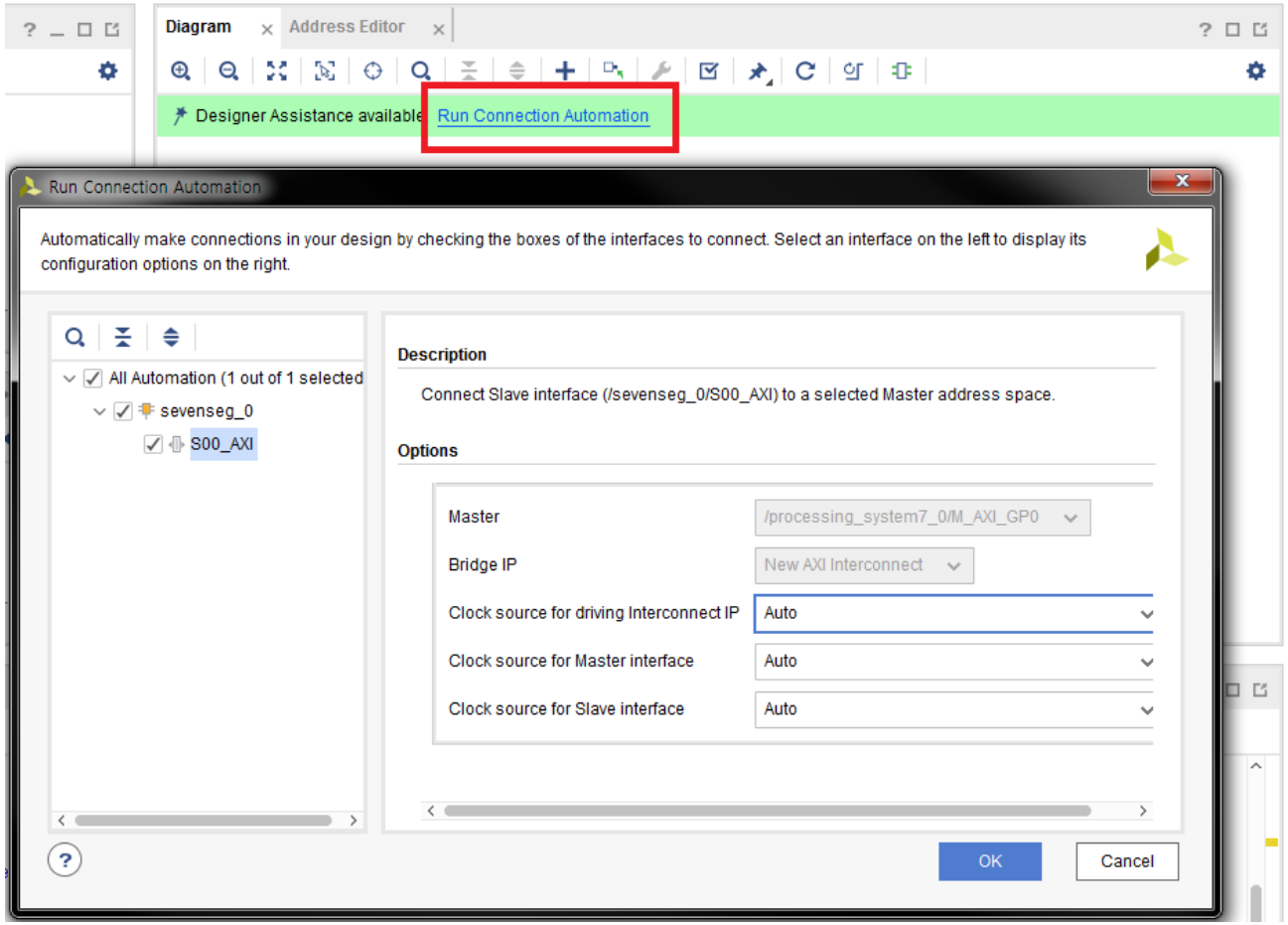
- Processing System과 AXI4 Peripheral 연결
  - ✓ Diagram 창에서 마우스 커서를 sevenseg_0 블록의 SEGDATA 출력 포트를 우클릭 해 Make External을 클릭한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

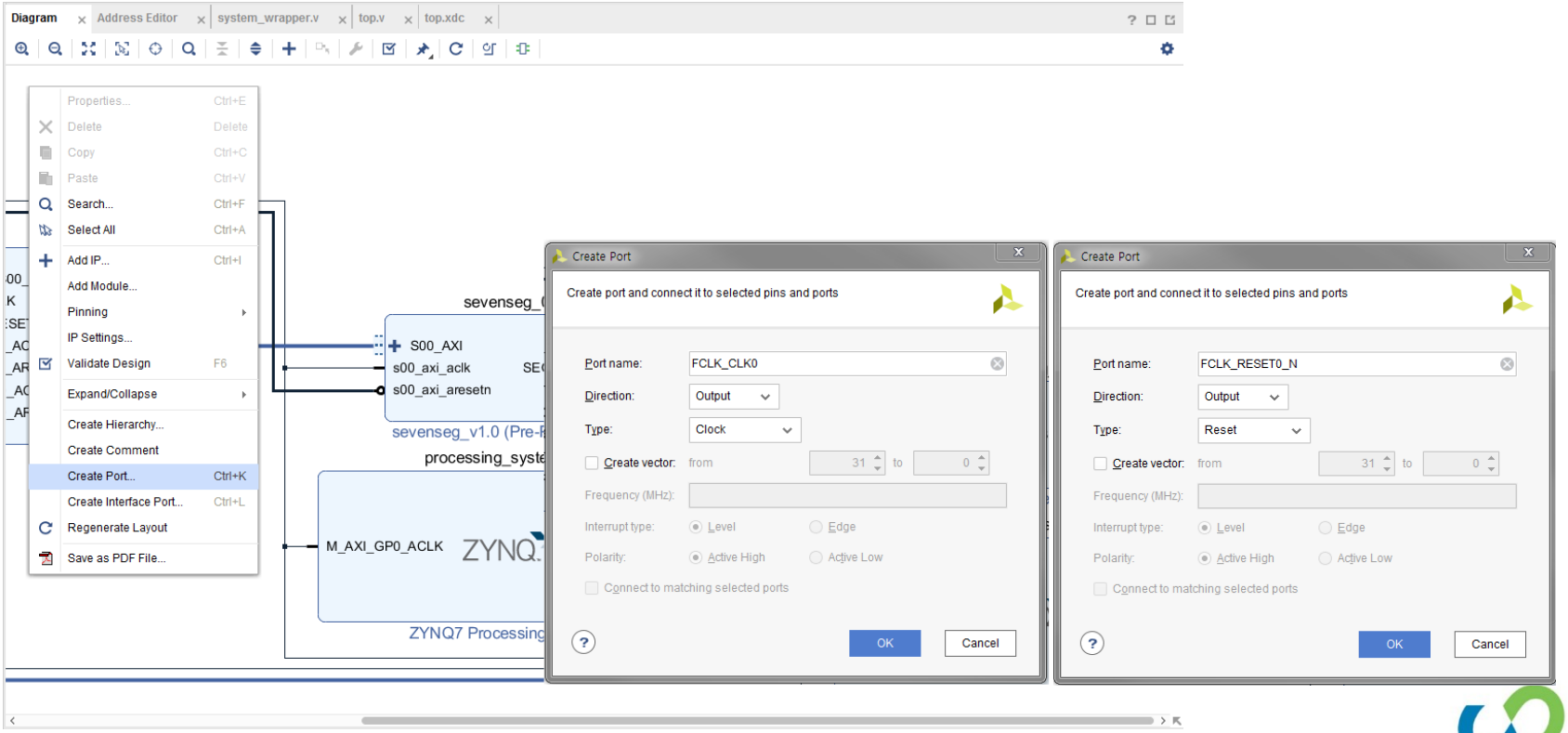
- Processing System과 AXI4 Peripheral 연결
  - ✓ Diagram 창에서 위의 Run Connection Automation을 클릭하여 실행한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

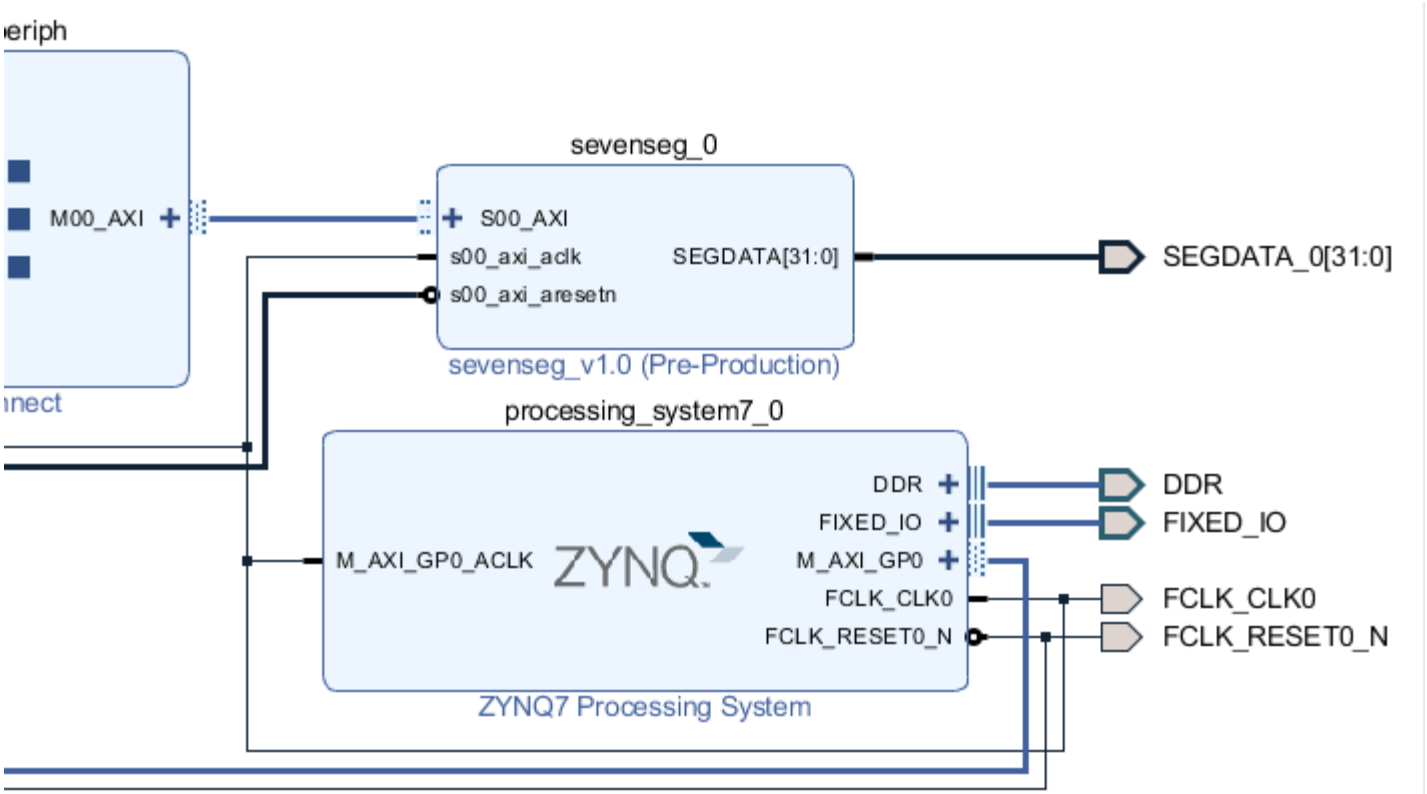
- Processing System과 AXI4 Peripheral 연결
  - ✓ Diagram 창에서 빈 곳에 우클릭 하여 Create Port를 통해 포트를 생성한다.
  - ✓ PS 영역에서 생성하는 클럭과 리셋을 사용하기 위해 다음과 같이 클럭과 리셋 포트를 출력 포트로 각각 생성한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

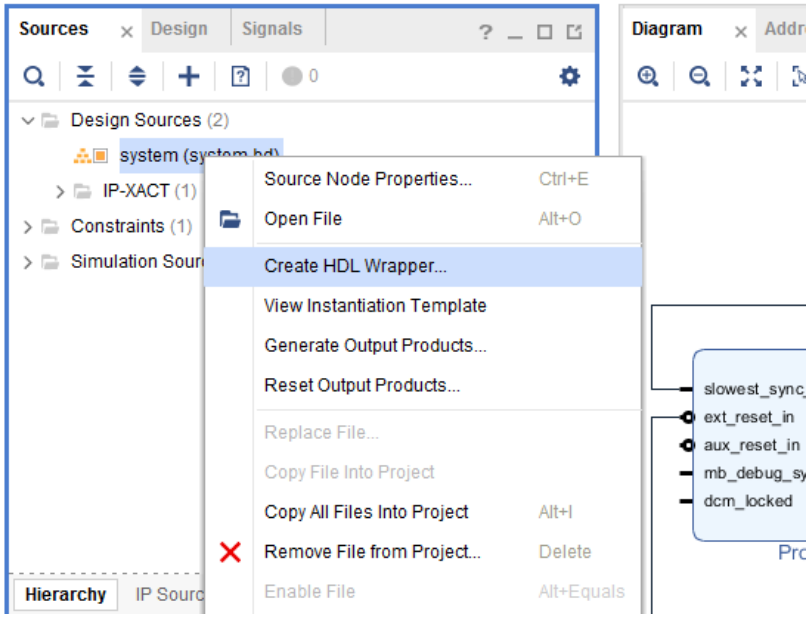
- Processing System과 AXI4 Peripheral 연결
  - ✓ 생성한 포트와 ZYNQ7 Processing System의 클럭, 리셋 출력을 연결한다.

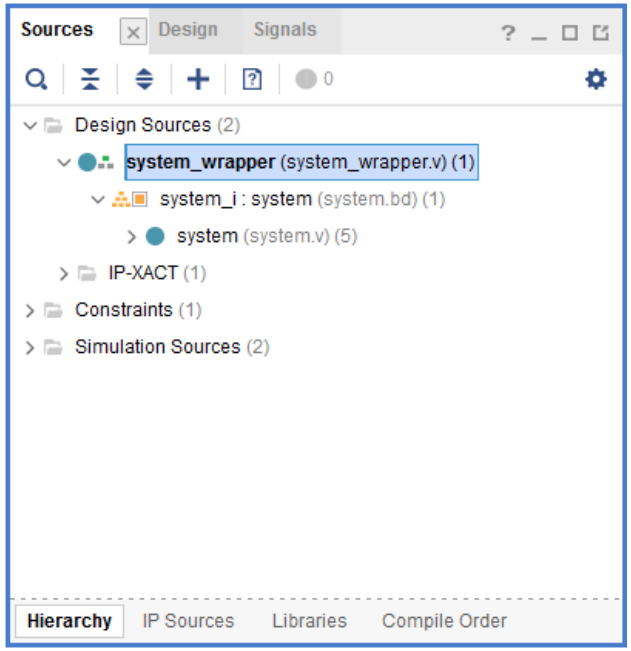


HUINS

ARM을 이용한 LED, 7-Segment 제어

- HDL System Wrapper 생성
  - ✓ Hierarchy 탭에서 system(system.bd)을 마우스 우클릭 하여 Create Wrapper를 실행한다.
  - ✓ 앞서 작성한 시스템 block design이 system_wrapper라는 하나의 모듈로 출력된다.

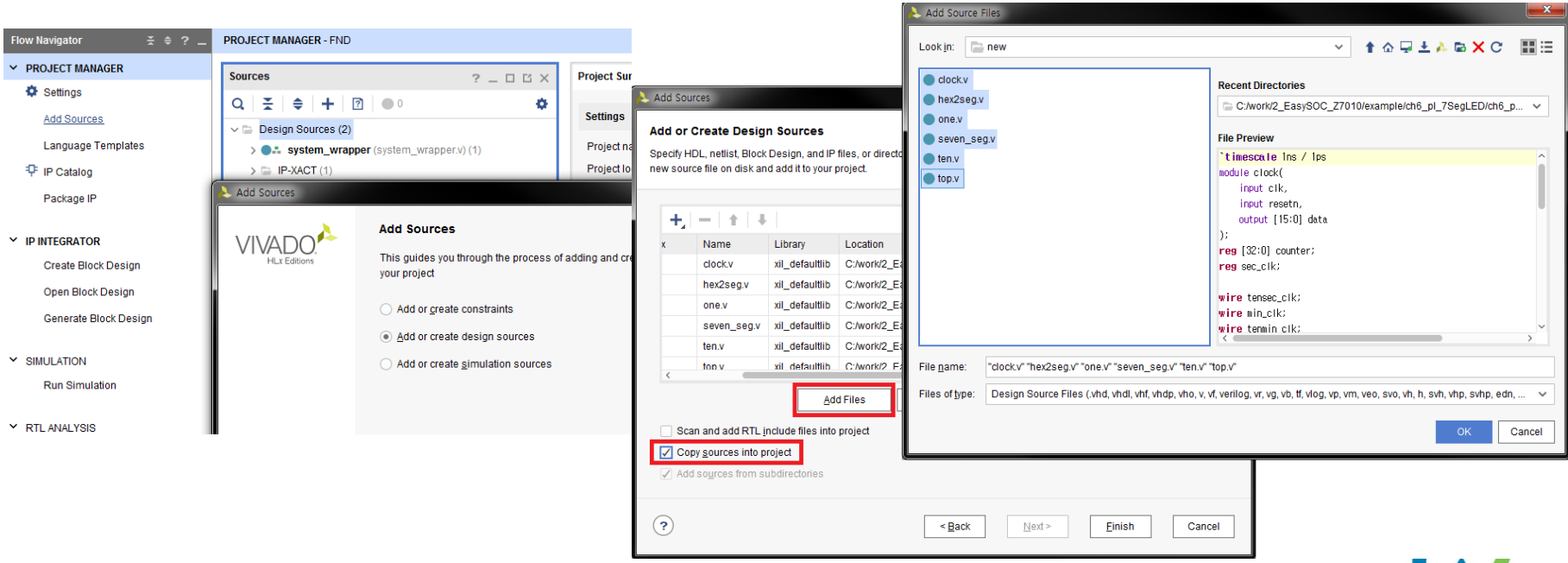




HUINS

ARM을 이용한 LED, 7-Segment 제어

- RTL 코드 수정
  - ✓ PROJECT MANAGER – Add Sources를 선택한다.
  - ✓ Add Files 버튼을 클릭해 'FPGA를 이용한 LED, 7-Segment 제어'에서 생성한 RTL 코드를 추가한다.
  - ✓ Copy Sources into project를 선택할 경우 사본을 생성해, 코드를 수정하더라도 기존의 프로젝트에 영향을 미치지 않는다.



ARM을 이용한 LED, 7-Segment 제어

- RTL 코드 수정
  - ✓ top.v 파일을 수정한다.
  - ✓ Processing System에서 시간 데이터를 생성할 예정이므로 top.v에서 clock 모듈은 더 사용하지 않아 instance를 주석으로 처리한다.

system_wrapper.v | top.v

C:/work/2_EasySOC_Z7010/example/ch7_ps_7SegLED/ch7_ps_7SegLED.srcs/sources_1/imports/new/top.v

```
1   `timescale 1ns / 1ps
2   module top(
3       input clk_in,
4       input resetn,
5       output [7:0] led_out,
6       output [7:0] seg_out,
7       output [3:0] segcom
8   );
9
10  wire [15:0] data;
11  //clock clock_u(.clk(clk_in), .resetn(resetn), .data(data));
12  seven_seg seven_seg_u(.clk_in(clk_in), .resetn(resetn), .data(data), .seg_out(seg_out), .com_out(segcom), .led_out(led_out));
13
14  endmodule
```

HUINS

ARM을 이용한 LED, 7-Segment 제어

- RTL 코드 수정
  - ✓ top모듈(top.v)에 system_wrapper 모듈의 인스턴스를 추가하고, 해당하는 입출력 포트를 선언한다.
  - ✓ system_wrapper의 출력인 SEGDATA_0는 기존의 wire인 [15:0]data에 연결한다.

```
40  system_wrapper system_wrapper_u(
41      .DDR_addr(DDR_addr),
42      .DDR_ba(DDR_ba),
43      .DDR_cas_n(DDR_cas_n),
44      .DDR_ck_n(DDR_ck_n),
45      .DDR_ck_p(DDR_ck_p),
46      .DDR_cke(DDR_cke),
47      .DDR_cs_n(DDR_cs_n),
48      .DDR_dm(DDR_dm),
49      .DDR_dq(DDR_dq),
50      .DDR_dqs_n(DDR_dqs_n),
51      .DDR_dqs_p(DDR_dqs_p),
52      .DDR_odt(DDR_odt),
53      .DDR_ras_n(DDR_ras_n),
54      .DDR_reset_n(DDR_reset_n),
55      .DDR_we_n(DDR_we_n),
56      .FCLK_CLK0(FCLK_CLK0),
57      .FCLK_RESET0_N(FCLK_RESET0_N),
58      .FIXED_IO_ddr_vrn(FIXED_IO_ddr_vrn),
59      .FIXED_IO_ddr_vrp(FIXED_IO_ddr_vrp),
60      .FIXED_IO_mio(FIXED_IO_mio),
61      .FIXED_IO_ps_clk(FIXED_IO_ps_clk),
62      .FIXED_IO_ps_porb(FIXED_IO_ps_porb),
63      .FIXED_IO_ps_srstb(FIXED_IO_ps_srstb),
64      .SEGDATA_0(data)
65      );
66  endmodule
```

HUINS

ARM을 이용한 LED, 7-Segment 제어

- RTL 코드 수정
  - ✓ top모듈에서 clk_in과 resetn를 삭제한다. system_wrapper 모듈의 클럭 FCLK_CLK0와 리셋 FCLK_RESET0_N을 대신하여 사용한다.
  - ✓ system_wrapper의 출력인 FCLK_CLK0, FCLK_RESET0_N, SEGDATA_0는 탑 모듈에서의 입출력이 아니므로 제외하고 입출력을 연결해준다(5번 – 30번).

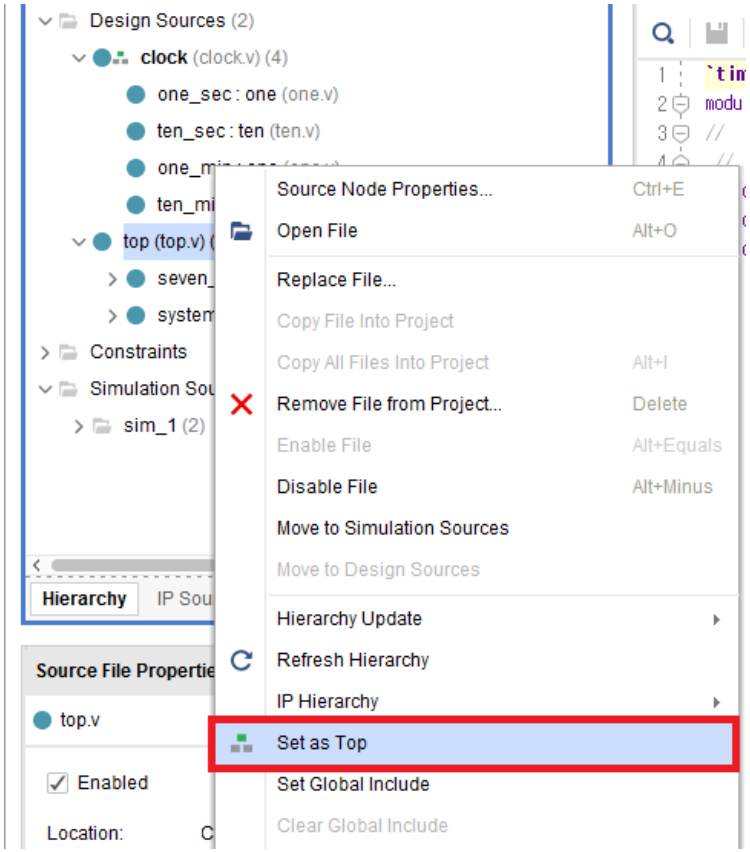
```
2   module top(
3   //      input clk_in,
4   //      input resetn,
5       output [7:0] led_out,
6       output [7:0] seg_out,
7       output [3:0] segcom,
8       inout [14:0]DDR_addr,
9       inout [2:0]DDR_ba,
10      inout DDR_cas_n,
11      inout DDR_ck_n,
12      inout DDR_ck_p,
13      inout DDR_cke,
14      inout DDR_cs_n,
15      inout [1:0]DDR_dm,
16      inout [15:0]DDR_dq,
17      inout [1:0]DDR_dqs_n,
18      inout [1:0]DDR_dqs_p,
19      inout DDR_odt,
20      inout DDR_ras_n,
21      inout DDR_reset_n,
22      inout DDR_we_n,
23  //      output FCLK_CLK0,
24  //      output FCLK_RESET0_N,
25      inout FIXED_IO_ddr_vrn,
26      inout FIXED_IO_ddr_vrp,
27      inout [31:0]FIXED_IO_mio,
28      inout FIXED_IO_ps_clk,
29      inout FIXED_IO_ps_porb,
30      inout FIXED_IO_ps_srstb
31  //      output [31:0]SEGDATA_0,
32  );
33  wire FCLK_CLK0;
34  wire FCLK_RESET0_N;
```

```
36  wire [15:0] data;
37  //clock clock_u(.clk(clk_in), .resetn(resetn), .data(data));
38  seven_seg seven_seg_u( clk_in(FCLK_CLK0), .resetn(FCLK_RESET0_N), .data(data), .seg_out(seg_out), .com_out(segcom), .led_out(led_out));
```

HUINS

ARM을 이용한 LED, 7-Segment 제어

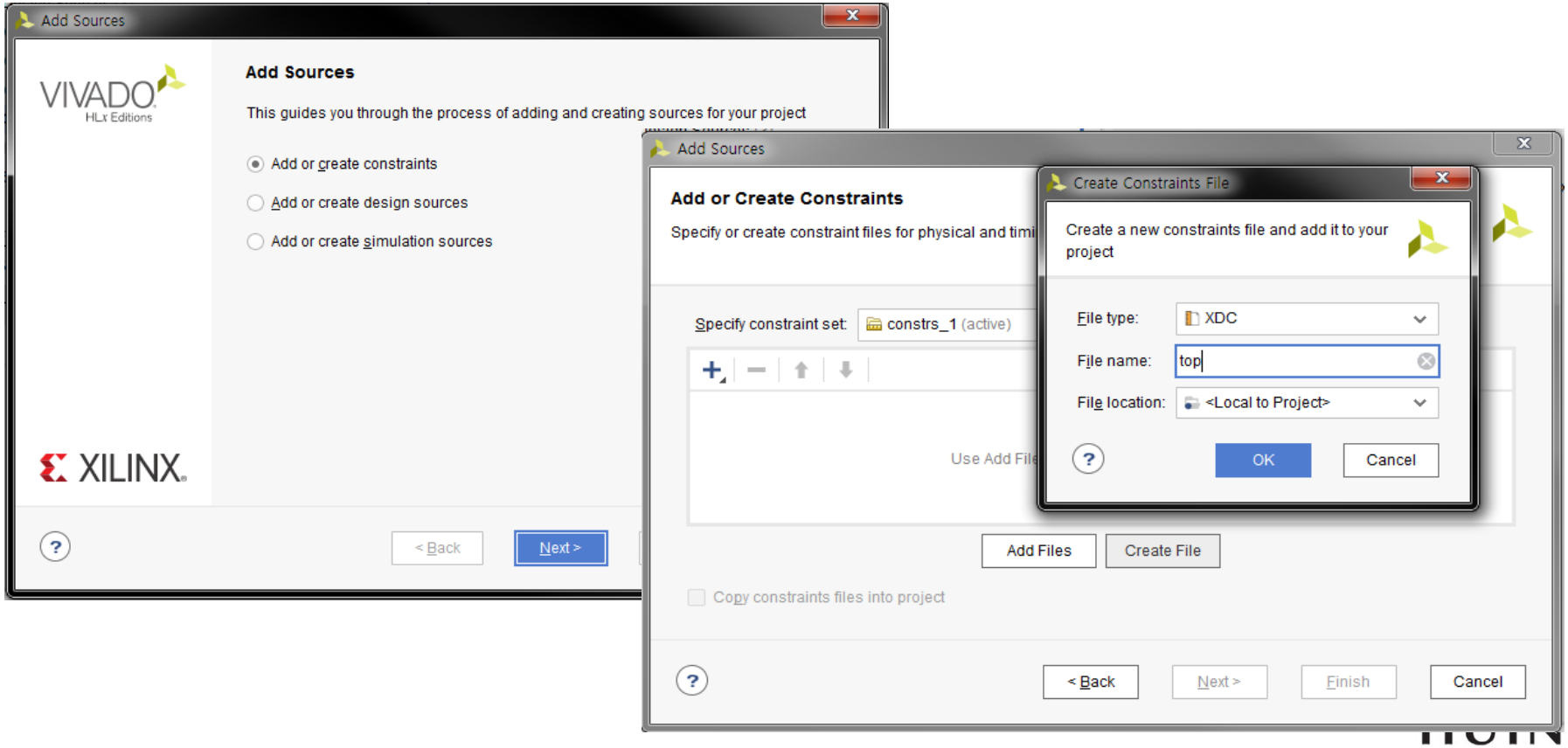
- RTL 코드 수정
  - ✓ top.v 파일을 우클릭하여 Set as Top을 선택해 Top 모듈로 설정한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

- Constraints 설정
  - ✓ Project Manager - Add Sources에서 "Add or create constrains" 항목을 선택한다.
  - ✓ Create File을 선택해 top.xdc 파일을 생성한다.



ARM을 이용한 LED, 7-Segment 제어

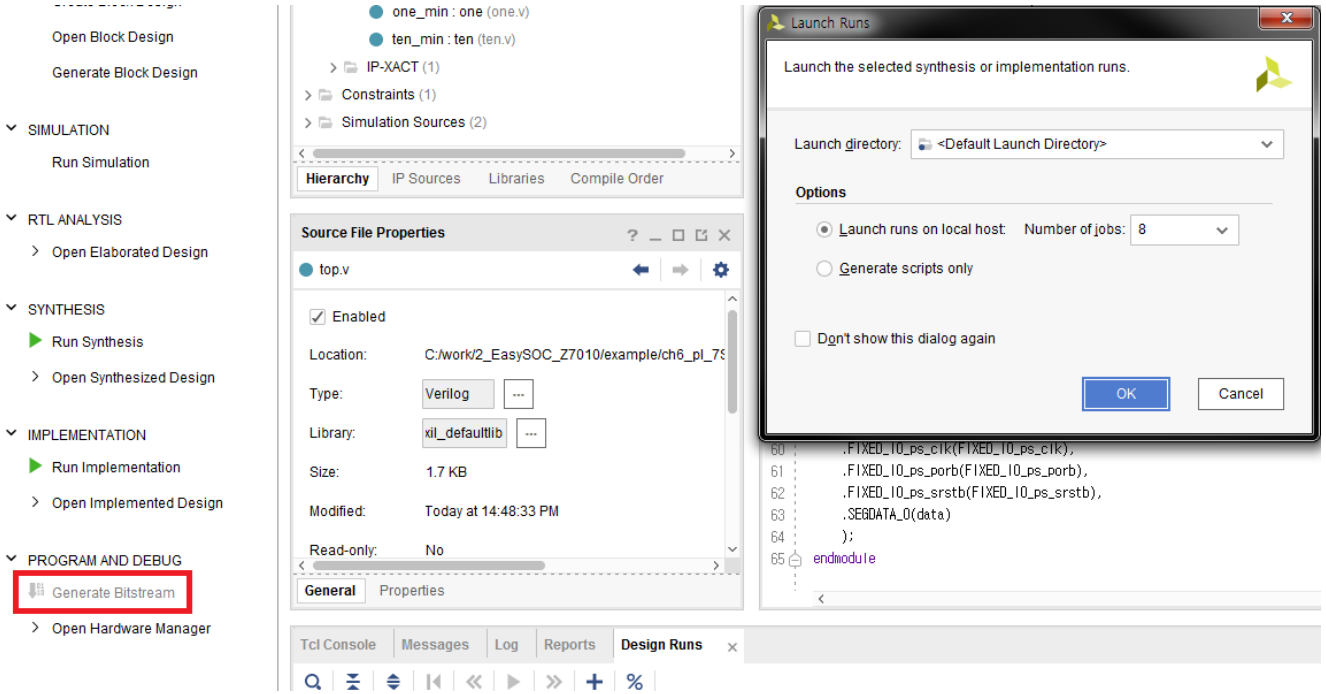
- Constraints 설정
  - ✓ top.xdc 파일의 내용을 다음과 같이 작성한다.

```
set_property IOSTANDARD "LVCMOS33" [get_ports "seg_out[*]"]
set_property PACKAGE_PIN "J14" [get_ports "seg_out[7]"]
set_property PACKAGE_PIN "H12" [get_ports "seg_out[6]"]
set_property PACKAGE_PIN "N14" [get_ports "seg_out[5]"]
set_property PACKAGE_PIN "R15" [get_ports "seg_out[4]"]
set_property PACKAGE_PIN "L14" [get_ports "seg_out[3]"]
set_property PACKAGE_PIN "L15" [get_ports "seg_out[2]"]
set_property PACKAGE_PIN "J13" [get_ports "seg_out[1]"]
set_property PACKAGE_PIN "H14" [get_ports "seg_out[0]"]

set_property IOSTANDARD "LVCMOS33" [get_ports "segcom[*]"]
set_property PACKAGE_PIN "M12" [get_ports "segcom[3]"]
set_property PACKAGE_PIN "R13" [get_ports "segcom[2]"]
set_property PACKAGE_PIN "M15" [get_ports "segcom[1]"]
set_property PACKAGE_PIN "J15" [get_ports "segcom[0]"]

set_property IOSTANDARD "LVCMOS33" [get_ports "led_out[*]"]
set_property PACKAGE_PIN "N13" [get_ports "led_out[7]"]
set_property PACKAGE_PIN "L13" [get_ports "led_out[6]"]
set_property PACKAGE_PIN "G11" [get_ports "led_out[5]"]
set_property PACKAGE_PIN "H11" [get_ports "led_out[4]"]
set_property PACKAGE_PIN "R12" [get_ports "led_out[3]"]
set_property PACKAGE_PIN "M14" [get_ports "led_out[2]"]
set_property PACKAGE_PIN "P15" [get_ports "led_out[1]"]
set_property PACKAGE_PIN "H13" [get_ports "led_out[0]"]
```

HUINS

ARM을 이용한 LED, 7-Segment 제어

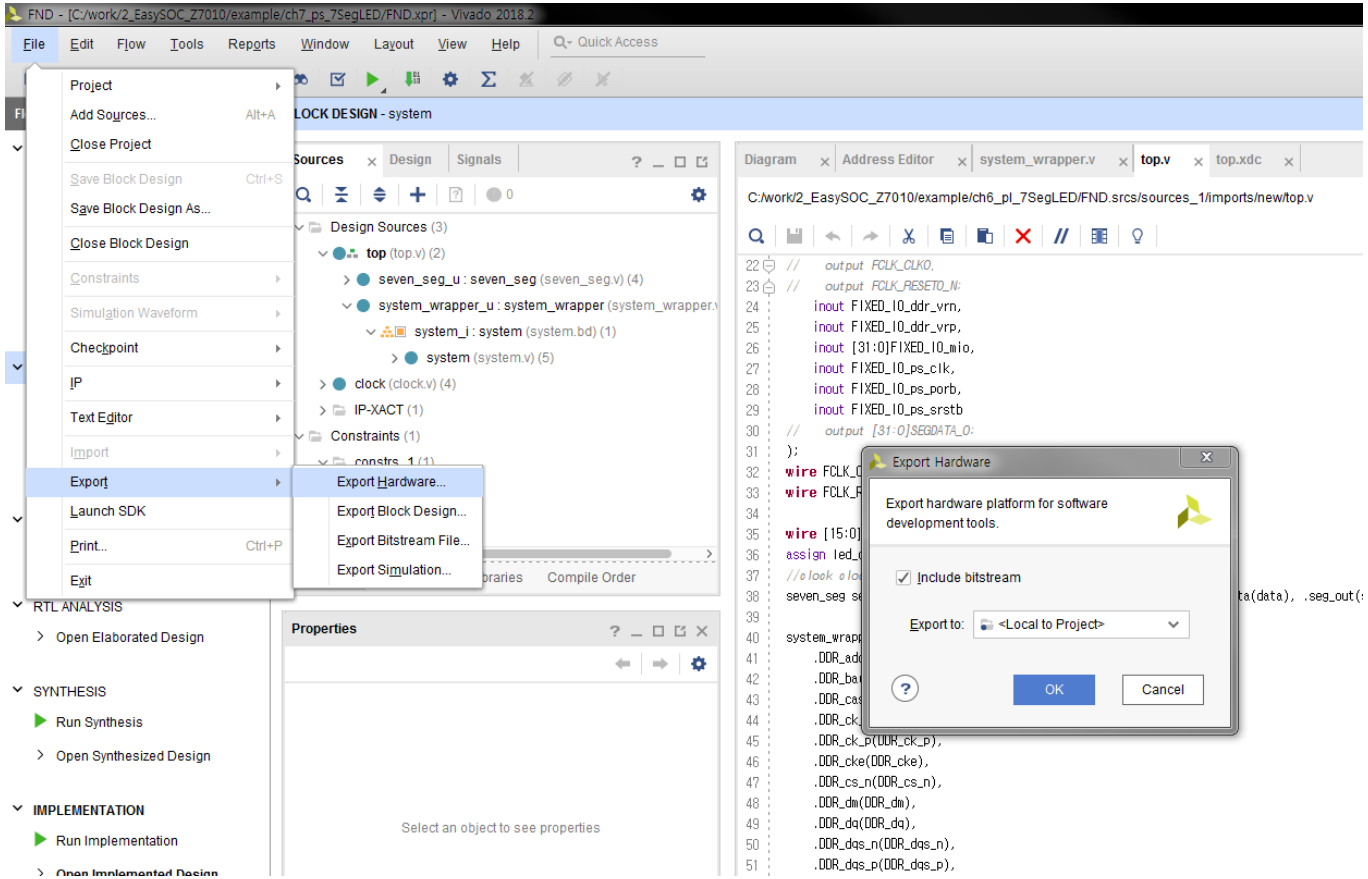
- Bitstream 파일 생성
  - ✓ Generate Bitstream을 클릭해 synthesis, implementation, generate bitstream을 실행한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

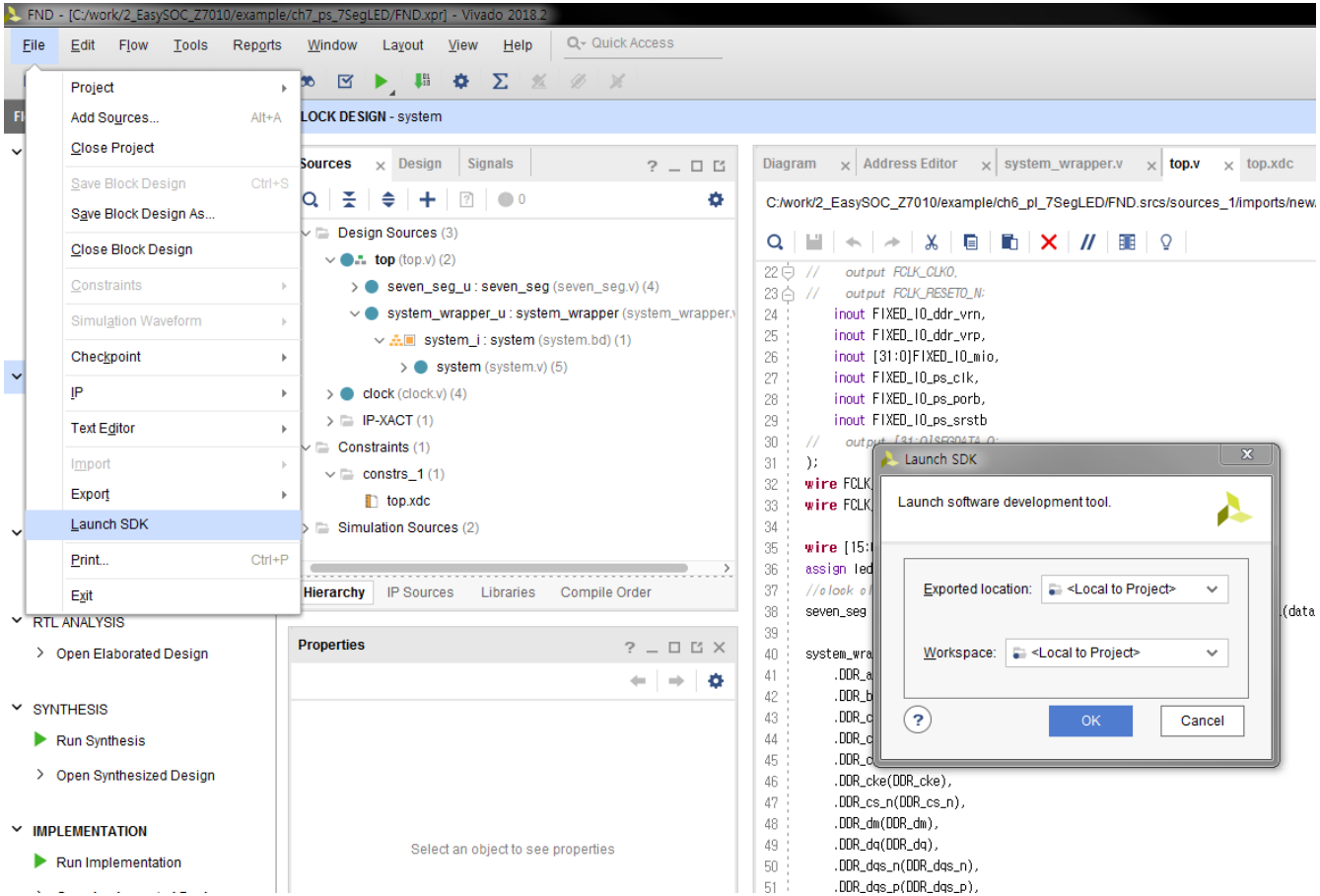
- Vivado 프로젝트 기반 SDK 프로젝트 생성
  - ✓ 메뉴 File – Export – Export Hardware 클릭 후 Include bitstream 항목을 체크하여 bit파일을 포함하여 H/W를 export한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

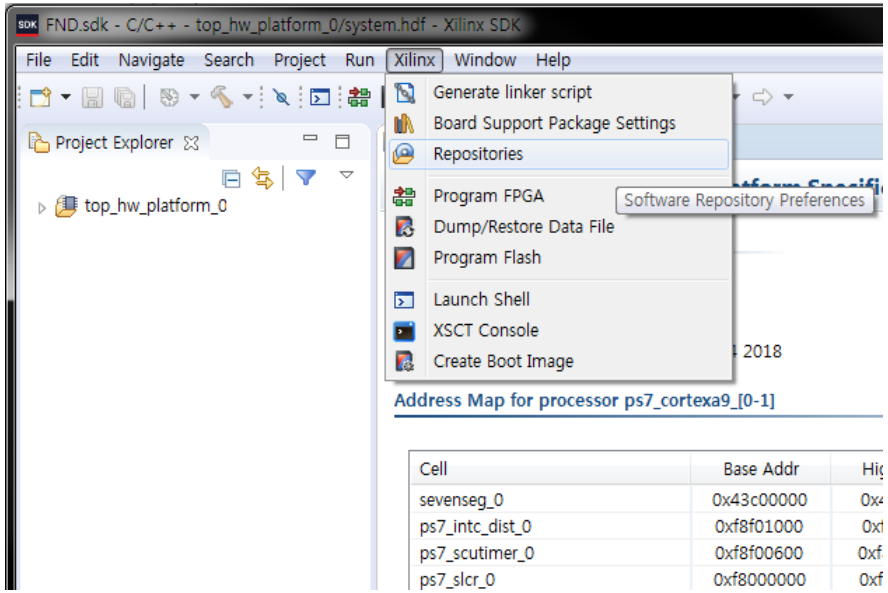
- Vivado 프로젝트 기반 SDK 프로젝트 생성
  - ✓ File – Launch SDK를 클릭해 H/W가 export된 경로와 SDK 프로젝트를 생성할 경로를 선택하여 실행한다.

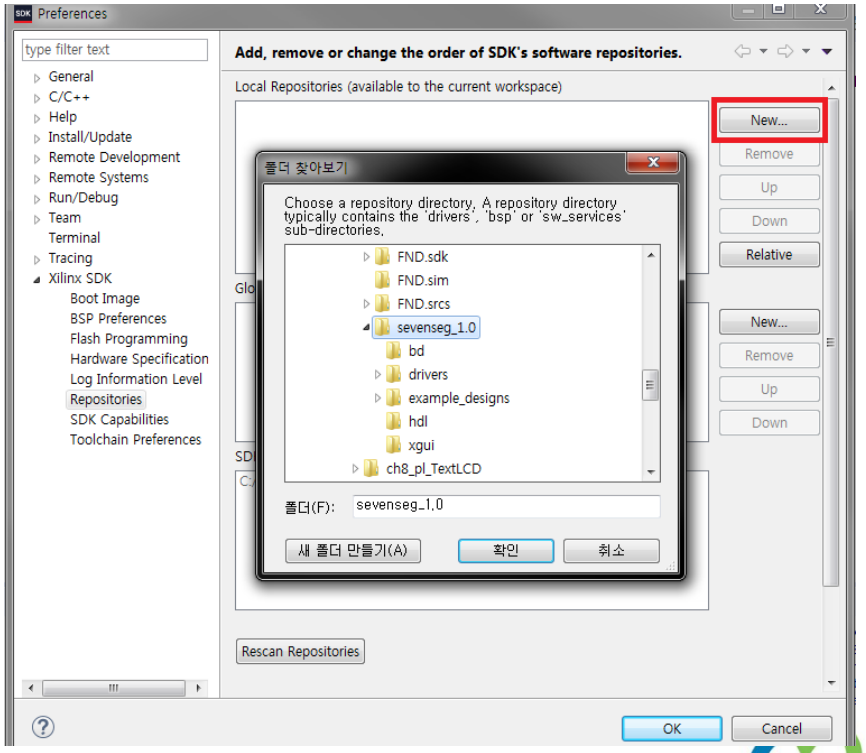


HUINS

ARM을 이용한 LED, 7-Segment 제어

- SDK Application 프로젝트 생성
  - ✓ sevenseg ip 드라이버를 추가하기 위해 Xilinx – Repositories를 선택한다.
  - ✓ Local Repositories 항목의 New 버튼 클릭 후, sevenseg IP 경로를 추가한다.

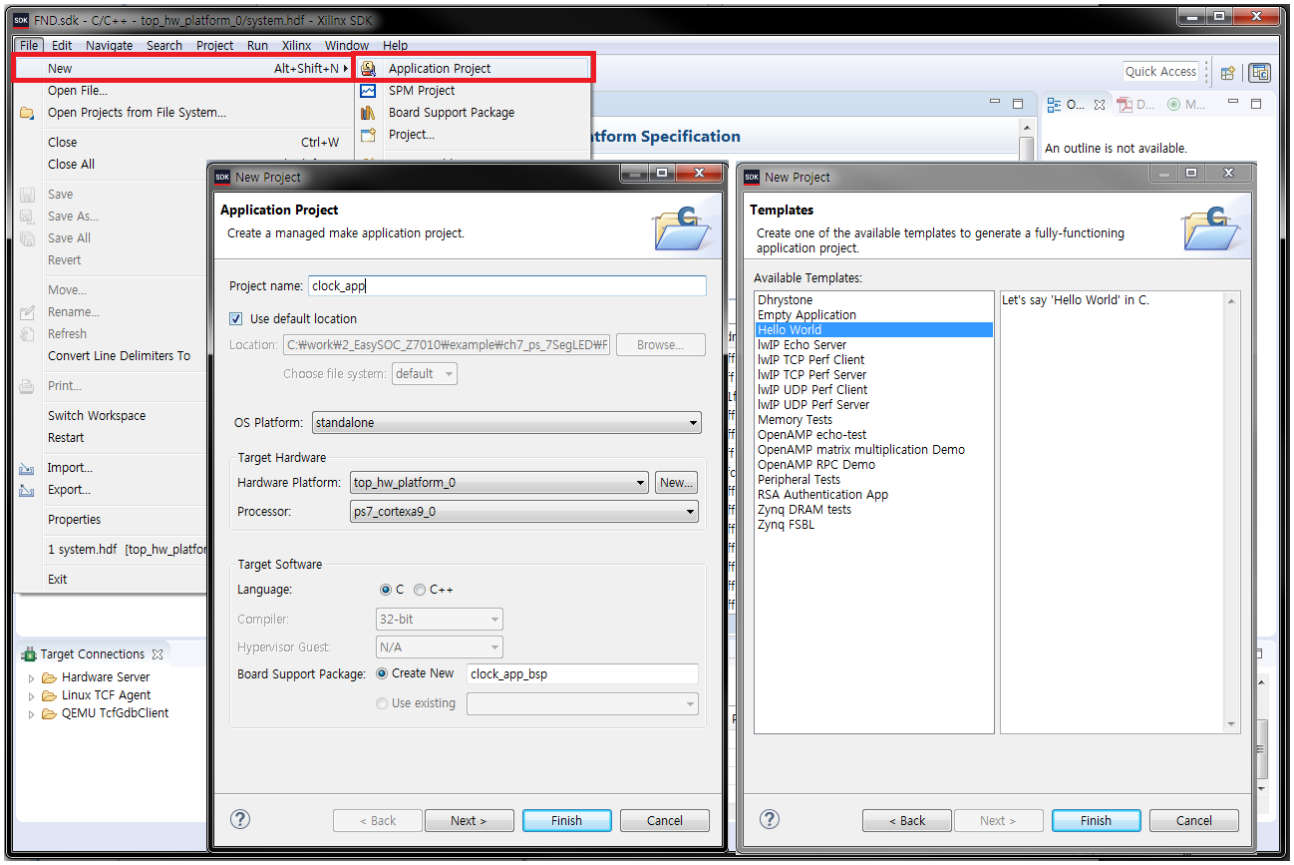




HUINS

ARM을 이용한 LED, 7-Segment 제어

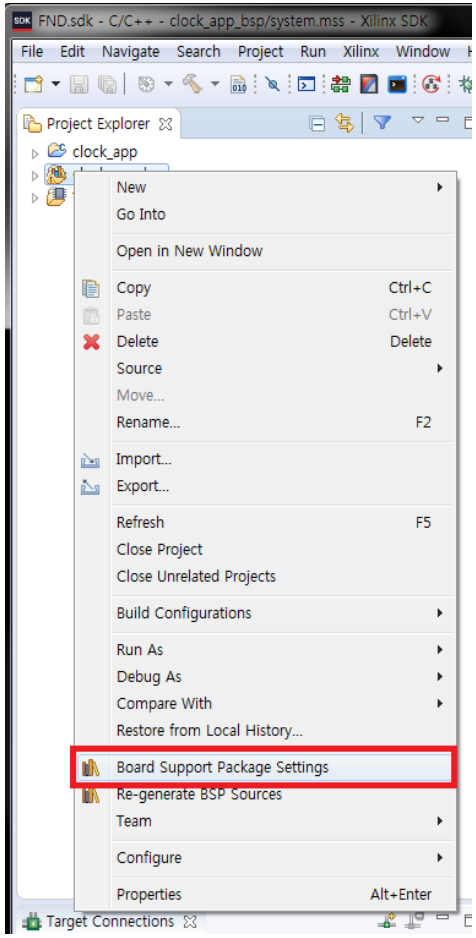
- SDK Application 프로젝트 생성
  - ✓ File – New – Application Project에서 프로젝트를 생성한다.
  - ✓ 프로젝트 이름을 정하고, Hello World 템플릿으로 프로젝트를 생성한다.

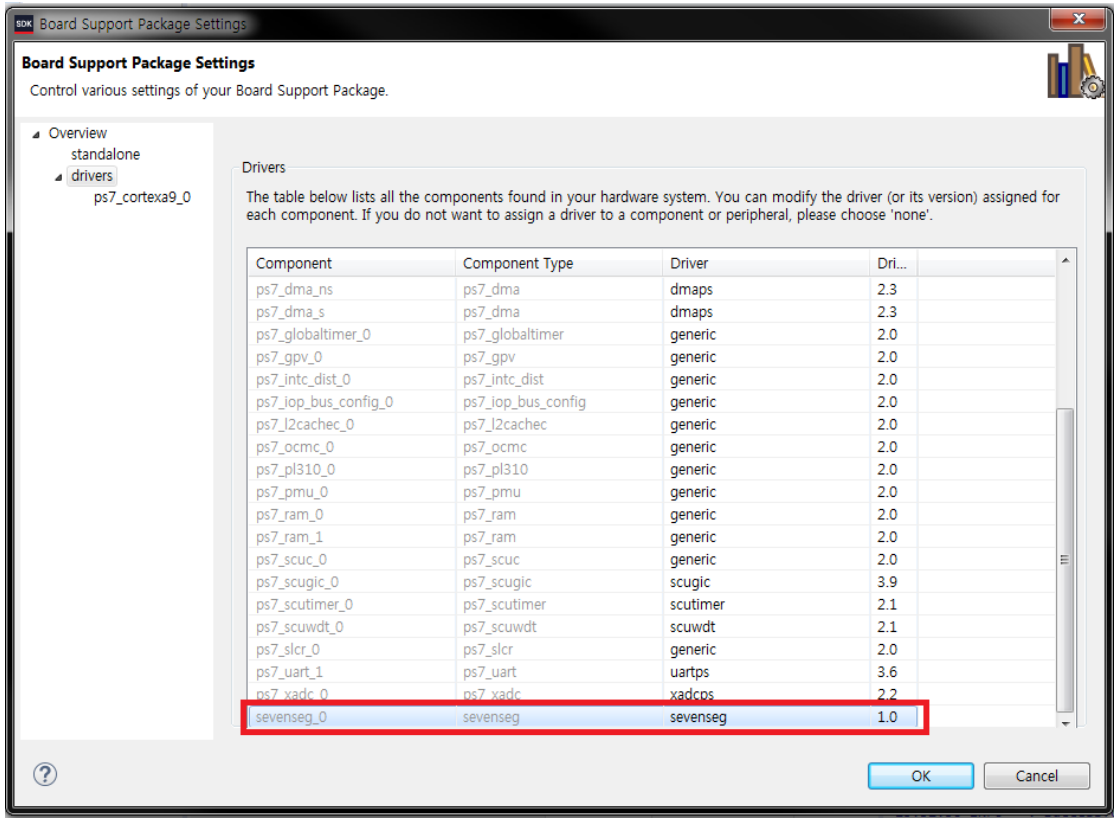


HUINS

ARM을 이용한 LED, 7-Segment 제어

✓ 생성된 clock_app_bsp를 우클릭하여 Board Support Package Settings로 진입할 경우 drivers 탭에서 sevenseg_0 ip를 확인할 수 있다.





HUINS

ARM을 이용한 LED, 7-Segment 제어

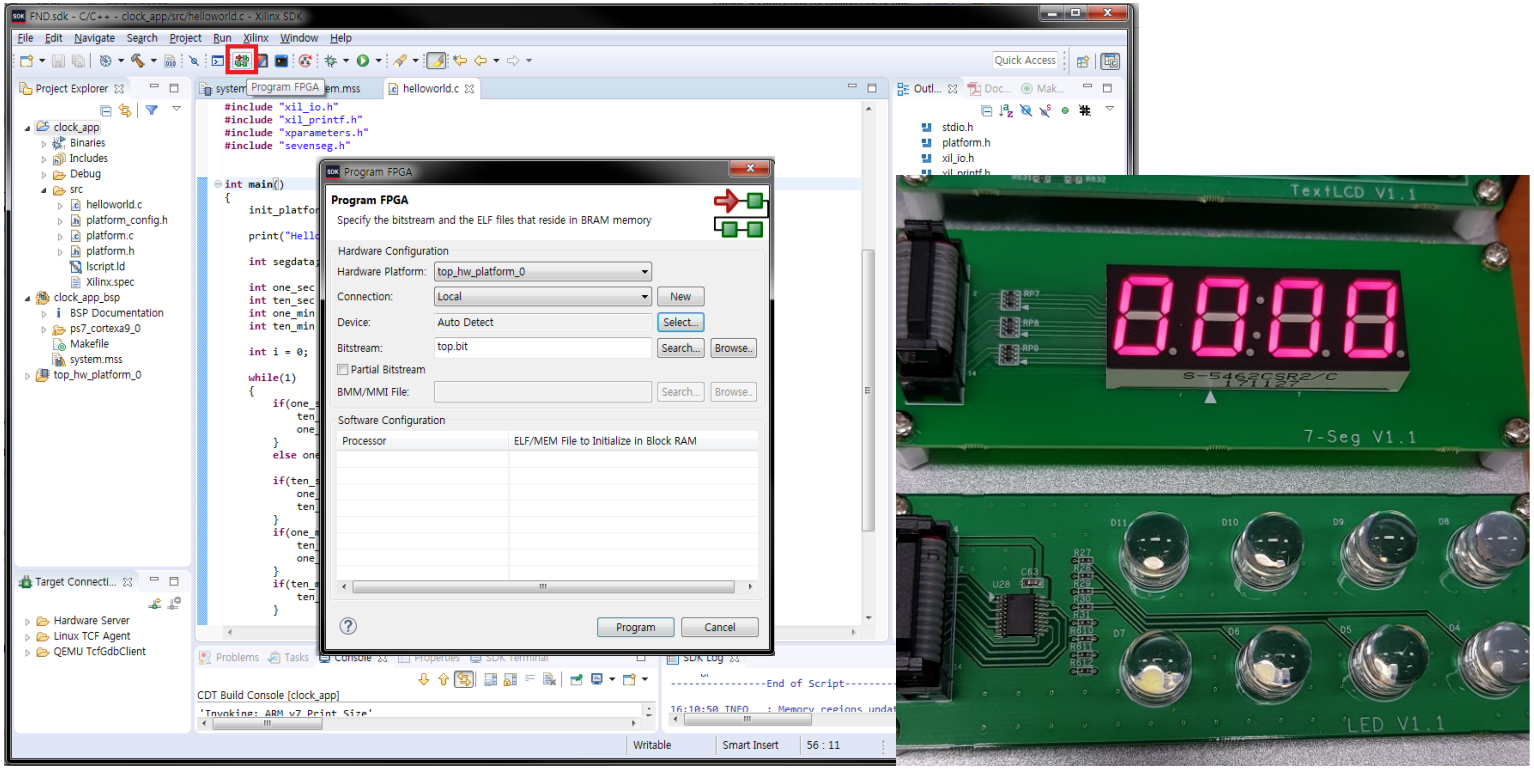
- Application C 코드 추가
  - ✓ clock_app – src – helloworld.c의 내용을 다음과 같이 수정한다.
  - ✓ 기존의 clock모듈을 대신하여 clock_app에서 초, 10초, 분, 10분의 데이터를 생성하며, 생성된 데이터를 sevenseg_0 드라이버의 메모리 주소에 write한다.

```
#include <stdio.h>
#include "platform.h"
#include "xil_io.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "sevenseg.h"


int main(){
    init_platform();

    print("Hello 60min₩n₩r");

    int segdata;

    int one_sec = 0;
    int ten_sec = 0;
    int one_min = 0;
    int ten_min = 0;

    int i = 0;
```

```
    while(1){
        if(one_sec == 9) {
            ten_sec++;
            one_sec=0;
        }
        else one_sec++;

        if(ten_sec == 6){
            one_min++;
            ten_sec=0;
        }
        if(one_min == 10){
            ten_min++;
            one_min=0;
        }
        if(ten_min == 6){
            ten_min=0;
        }

        for(i = 0; i < 0x6000000; i++);

        segdata = (ten_min << 12) + (one_min << 8) + (ten_sec << 4) + one_sec;

        SEVENSEG_mWriteReg (XPAR_SEVENSEG_0_S00_AXI_BASEADDR, 0, segdata);
    }

    cleanup_platform();
    return 0;
}
```

ARM을 이용한 LED, 7-Segment 제어

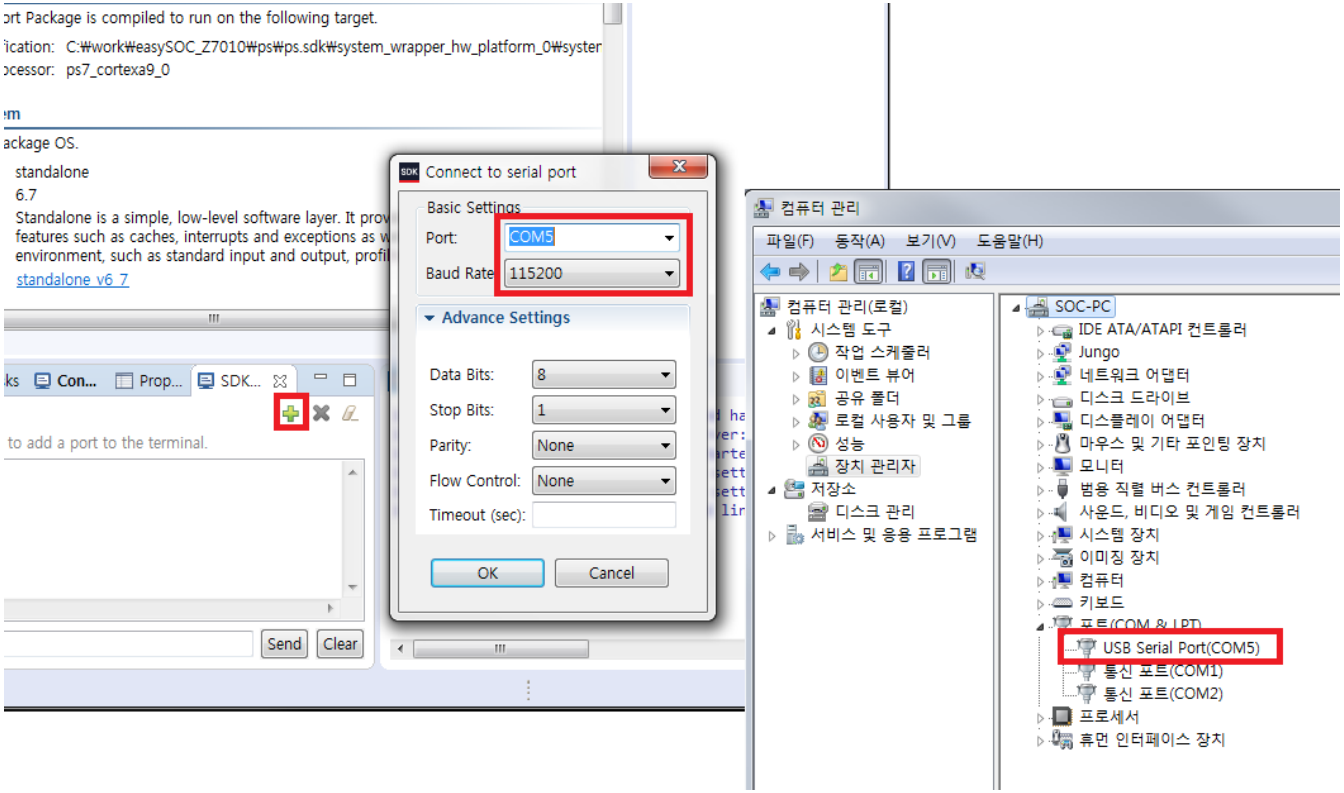
- ✓ Program FPGA 버튼을 클릭하여 H/W와 함께 export 된 bitstream 파일을 Zynq-7010의 PL 영역에 다운로드 한다.
- ✓ 다운로드가 완료되면 보드의 7-Segment에 0000이 표시된다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

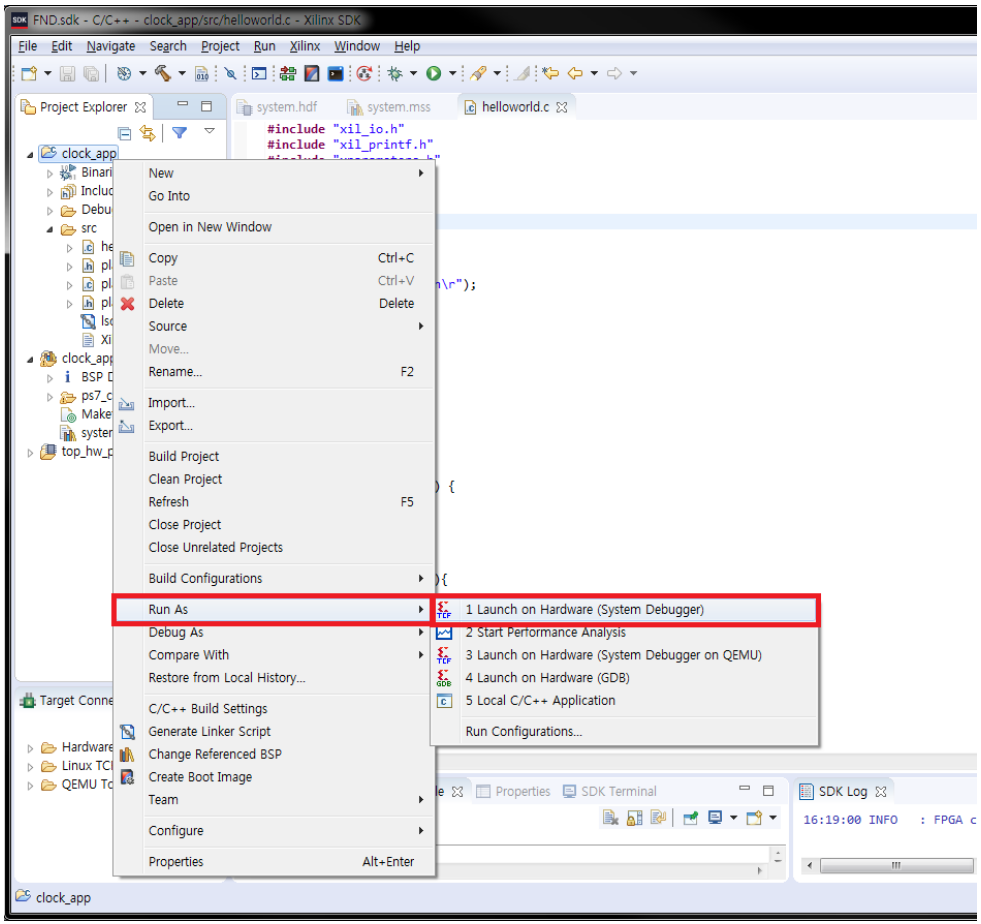
- ✓ SDK 창 하단에 SDK Terminal을 선택하여 서브 창을 연 후, 연결 버튼을 시리얼 포트 연결 창을 실행한다.
- ✓ 장치관리자에서 확인한 USB Serial Port에 맞게 포트를 설정해 연결한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

✓ Firmware를 Cortex-A9에서 동작시키기 위해, SDK의 clock_app을 우클릭하여 Run As – Launch on Hardware (System Debugger) 항목을 실행한다.



HUINS

ARM을 이용한 LED, 7-Segment 제어

- ✓ EasySoC-Z7010의 7-Segment에 시간이 출력된다.
- ✓ SEVENSEG_mWriteREG는 Xil_Out32() 함수의 매크로이며, 7-Segment에 데이터를 출력한다.
  - parameter는 (BaseAddress, RegOffset, Data)이다.
  - (BaseAddress + RegOffset)의 주소에 (u32)Data를 write한다.
  - BaseAddress는 driver-ip의 base address인 0X43C00000이다.

```
/* Definitions for peripheral SEVENSEG_0 */
#define XPAR_SEVENSEG_0_DEVICE_ID 0
#define XPAR_SEVENSEG_0_S00_AXI_BASEADDR 0x43C00000
#define XPAR_SEVENSEG_0_S00_AXI_HIGHADDR 0x43C0FFFF
```

  - Offset의 값에 따라 AXI를 통해 write 할 register를 선택할 수 있다.
    - » RegOffset이 0일 경우 BaseAddress이므로 slv_reg0에 입력된다.

```
236        // Slave register 0
237        slv_reg0[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
```

HUINS

ARM을 이용한 LED, 7-Segment 제어

✓ 7-segment IP에서 data 값에 따라 led와 7-segment에 출력한다.

HUINS

감사합니다.

HUINS